

AI-Assisted Guest Support Assistant Documentation

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

Table of Contents

Document 1: Project Plan

Introduction — 4 Project Overview — 4 2.1 Scope — 4 2.1.1 Identification — 4 2.1.2 System Overview — 5 2.1.3 Document Overview — 5 2.2 Current System or Situation — 5 2.3 Background, Objectives, and Scope — 6 2.4 Operational Policies and Constraints — 6 2.5 Users or Involved Personnel — 6 Development Background and Approach — 6 3.1 High Level Estimates — 6 3.2 Key Contacts and Stakeholders — 7 Features, Primary Deliverables, and External Commitments — 7 4.1 Feature List — 7 4.2 Customer Deliverables — 8 Project Schedule — 8 5.1 Major Project Milestones — 8 Project Work and Product Estimates — 9 6.1 Estimate Summary — 9 Project Resource Requirements — 9 7.1 Staffing and Skill Requirements — 9 7.2 Plan to Fill Skill Gaps — 9 Dependencies and Constraints — 10 8.1 Constraints — 10 Risk Management — 10 9.1 Risk Management Strategy — 10 9.2 Initial Risk List — 11 Project Configuration and Data Management — 12 10.1 Configuration Management — 12 Project Process — 12 11.1 Software Life Cycle Model — 12 Glossary — 13 Change Record — 13

Document 2: Requirements Specification (SRS)

1. Introduction — 17 1.1 Purpose — 17 1.2 Scope — 17 1.3 Definitions, Acronyms, and Abbreviations — 17 1.4 Overview of the Document — 18 1.5 References — 18 1.6 Assumptions — 18 1.7 Constraints — 19 1.8 Explicit Exclusions — 19 1.9 Questions — 19
2. Overall Description — 20 2.1 Product Perspective — 20 2.2 Product Functions — 21 2.3 User Classes and Characteristics — 21 2.4 Design and Implementation Constraints — 21 2.4.1 Supported Vs Unsupported Question Taxonomy — 21 2.5 Assumptions and Dependencies — 22
3. System Interfaces — 22 3.1 System Interface — 22 3.2 User Interface — 22 3.3 Hardware Interfaces — 23 3.4 Software Interfaces — 23
4. Requirements — 23 4.1 Functional Requirements (FR-01 through FR-21) — 23 4.2 Non-Functional Requirements (NFR-01 through NFR-10) — 26
5. Design Considerations — 27 5.1 Software Design — 28 5.2 Database Design — 28 Conclusion — 28 Appendix — 30 Change Log — 30

Document 3: Design Document (SDD)

1. Introduction — 34 1.1 Purpose — 34 1.2 Scope — 34 1.3 Overview — 34 1.4 Definitions and Acronyms — 35 1.5 References — 35
2. System Overview — 36 2.1 Architectural Design — 36 2.2 Technical Design — 37 2.3 Design Rationale — 37 2.4 Architectural Views — 37 2.5 Architectural Assumptions — 38

3. Detailed Design — 38 3.1 Decomposition Description — 38 3.2 Dependency Description — 39 3.3 Interface Description — 39
4. User Interface Design — 40 4.1 UI Design — 40
5. Database and Data Design — 41
6. API Integration Plan — 42 6.1 Overview — 42 6.1 Core Integration – Conceptual — 42 6.2 Evaluation Metrics — 44
7. Appendices — 46
8. Change Log — 46

Document 4: Test Plan & Results

How to Use This Document — 49 Test Environment — 50 FR-01: Simple Entry Point — 50 FR-02 / FR-21: Reservation Validation & Unauthorized Access Prevention — 50 FR-03: Conversational UI — 51 FR-04 / FR-06 / FR-12: Human Support & Escalation — 52 FR-07: Intent Classification — 52 FR-08: User Context-Aware Responses — 53 FR-09 / FR-10: Hallucination Minimization & Authoritative Data — 53 FR-11 / FR-14 / FR-19: Confidence Evaluation & Logging — 54 FR-16 / FR-17: Multi-Turn Context & Session Timeout — 54 FR-20: Financial Operation Restriction — 55 NFR-01: Accuracy — 55 NFR-02: Response Time — 55 NFR-09: Hallucination Rate — 56 NFR-08: Security — 56 Test Summary — 57

Document 5: Developer User Manual

1. Introduction — 59
2. System Requirements — 60
3. Installation & First-Time Setup — 61 3.1 Clone the Repository — 61 3.3 Configure Environment Variables — 62 3.4 Load the Call Center Manual (RAG Setup) — 62 3.5 Set Up the Frontend — 62
4. Running the Application — 64 4.1 Start the Backend — 64 4.2 Start the Frontend — 64 4.3 Open the Application — 64
5. Using the Chatbot — 66 5.1 Asking a General Question — 66 5.2 Looking Up a Reservation — 67 5.3 Out-of-Scope Questions — 67 5.4 Session Timeout — 68
6. Viewing Logs and Sessions — 69 6.1 Terminal Logs — 69 6.2 Context Used Log — 69 6.3 Sessions Endpoint — 69
7. Test Data — 71
8. StreamlineVRS Connection — 72 8.1 Testing the Connection — 72 8.2 Monthly Credential Rotation — 72
9. Updating the Call Center Manual — 73
10. Troubleshooting Common Errors — 74
11. Security Checklist — 75

Project Plan for an AI Assistant for Vacations for YOU

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

Project Plan for an AI Assistant for Vacations for YOU	1
Introduction	3
Project Overview.....	3
2.1 Scope	3
2.1.1 Identification	3
2.1.2 System Overview.....	4
2.1.3 Document Overview	4
2.2 Current System or Situation.....	4
2.3 Background, Objectives, and Scope	4
2.4 Operational Policies and Constraints	5
2.5 Users or Involved Personnel.....	5
Development Background and Approach	5
3.1 High Level Estimates	5
3.2 Key Contacts and Stakeholders	6
Features, Primary Deliverables, and External Commitments	6
4.1 Feature List	6
4.2 Customer Deliverables.....	7
Project Schedule.....	7
5.1 Major Project Milestones.....	7
Project Work and Product Estimates.....	7
6.1 Estimate Summary.....	8
Project Resource Requirements.....	8
7.1 Staffing and Skill Requirements.....	8
7.2 Plan to Fill Skill Gaps	8

Dependencies and Constraints	8
8.1 Constraints	9
Risk Management	9
9.1 Risk Management Strategy	9
9.2 Initial Risk List	9
Project Configuration and Data Management	9
10.1 Configuration Management	9
Project Process	9
11.1 Software Life Cycle Model	10
Glossary	10
Change Record	10

Introduction

Vacations for You manage a high-volume vacation rental operation and processes a large number of guests inquires through a traditional call center model. This project focuses on the development of a functional Proof-of-Concept system capable of handling common guest inquiries such as check-in/check-out times, reservation details, cancelation policies, property amenities and other routine informational requests. The product will be a web based, AI assisted support chat system to help all guests and potential guests. The system is indented to reduce call-center load by answering common questions and providing a clear escalation path when needed.

Project Overview

2.1 Scope

The document specifies the project plan for the Phase 1 proof of concept AI-assisted guest support system for Vacations for YOU. The system outlined is designed to support guests with existing reservations by providing a fully conversational chatbot experience that delivers automated responses to common post-booking questions through a web-based chat interface.

The Phase 1 system will employ a large language model (LLM) to generate responses based on read only reservation, property, and policy data from the StreamlineVRS property management platform or an approved sandbox environment. The AI assistant will provide informative responses and present clear escalation guidance when questions fall outside of the scope or confidence threshold.

The project excludes booking creation or sales, payment processing, service order placement including but not limited to maintenance, account creation, or authentication beyond what is necessary to provide reservation information. The project is intentionally structured to validate feasibility, usability, and call-deflection potential for future channel expansion.

2.1.1 Identification

System Name: Vacations for You AI Support Assistant

Version: 1.0

Document Type: Project Plan
Sponsor: Vacations for You

2.1.2 System Overview

The AI-assisted guest support chat platform will follow a modular layered, client-server architecture. This separation of components improves maintainability and scalability while remaining appropriate for the POC implementation.

The architecture of the platform consists of:

- lightweight web-based frontend using React
- Python-based backend service using FastAPI
- AI processing layer utilizing a Large Language Model (LLM)
- Integration layer responsible for secure communication with the StreamlineVRS API.

The front-end provides the conversational UI through which guests interact, submit questions and responses. Communication to the back end will be done through HTTPS REST Endpoints. All integration with the StreamlineVRS API is strictly read-only to prevent reservation data from being modified by the system. The backend service will manage session handling and reservation validation. The AI processing component will implement a Retrieval-Augmented Generation (RAG) pattern, through which relevant reservation and property data are retrieved from StreamlineVRS prior to constructing the prompts. These prompts will then be submitted to an external LLM provider, such as OpenAI, to generate NL responses relevant to the guest's inquiries.

2.1.3 Document Overview

This document serves to outline a structured framework for managing and executing the project from initiation through deployment. It defines the project's scope, objectives, timelines, resources, and risks. It serves as a baseline reference for stakeholders and project team members throughout the lifecycle of the system.

2.2 Current System or Situation

Currently, Vacations for You relies on a traditional call center model to handle guest inquiries. This approach requires significant human staffing resources and results in high operation overhead. Many of these inquiries involve repetitive informational questions that could potentially be automated.

2.3 Background, Objectives, and Scope

With the rise of AI, using automated chat bot agents has proven itself for simple repetitive tasks. Several companies are already utilizing AI assistants to quickly provide customers with information and take a hefty workload off of human agents. This project is meant to provide a simpler and more efficient form in which customers can obtain information from Vacations for You.

Objectives:

- Develop POC AI chatbot
- Demonstrate ability to retrieve and present accurate reservation data
 - Validate chatbot efficiency
- Reduce call-center workload
- Offer 24/7 self-service support

2.4 Operational Policies and Constraints

The system must operate under the following constraints:

- No modification of customer data
- No handling of monetary transactions
- Unsupported questions must be escalated to human agent
- All API access must be read-only
- All communication must be secure
- Sensitive customer data must not be stored.

2.5 Users or Involved Personnel

- Guest with existing reservations
- Potential guest seeking property/area information
- Call center representatives
- Project sponsor
- Project development team

Development Background and Approach

3.1 High Level Estimates

The project will be completed within one academic semester and be divided into phases seen in the table below:

Phase	Duration
Requirements & Design	2 weeks

Prototype Development	5 weeks
Testing & QA	2 weeks
Refinement	1 week
Finalization	1 week

3.2 Key Contacts and Stakeholders

- Team Lead: Responsible for all external communication with stakeholders and keeping the development team on track.
- Team Members: Responsible for frontend, backend, and testing assigned tasks.
- Project Sponsor: Provides initial requirements, validation, and feedback on product development.

Features, Primary Deliverables, and External Commitments

4.1 Feature List

The phase one POC AI support assistant will include the following core features:

1. Web-Based Chat Interface
 - Embedded chat widget accessible from any page on existing website
2. Reservation Validation
 - Reservation lookup using reservation ID and/or Last Name
 - Read-long validation against StreamlineVRS or approved sandbox
3. AI-Powered Conversational Responses
 - Natural language understanding of guest inquires and context aware responses based and validated reservation data
 - Multi-turn conversational support
4. Retrieval Augmented Generation (RAG)
 - Dynamic retrieval of relevant reservation, property, and policy data
 - Prompt construction using retrieved data
5. Escalation & Human Handoff
 - Confidence based escalation mechanism
 - Clear instructions for contacting human support
 - Logging of escalation events
6. Conversation Logging
 - Session logging of guest questions and system responses
 - Tracking of unsupported inquires and escalation

4.2 Customer Deliverables

The final deliverables will include:

- Working web-based POC AI chatbot
- Technical documentation
 - Project Plan, SRS, SDD, User Manual
- Evaluation report and metrics
- Presentation and demonstration
- GitHub repository

Project Schedule

5.1 Major Project Milestones

This project is broken into three major milestones that we have defined as follows:

- Milestone One: Documentation Phase ---- 2/18/26
 - During this phase we are focusing on developing Requirements Documentation, and Design Documentation. Our focus will be requirements elicitation and developing a design based on the requirements gathered. Once both documents are approved, we will move forward into basic API implementation using OpenAI to set up milestone two.
- Milestone Two: Prototype Development ----- 3/25/26
 - After completing milestone two, our goal will be to create a working chatbot that can answer the basic questions needed to work as proof of concept. As an initial prototype, we expect to focus on the most basic implementation, and plan on expanding based on the success of the initial chatbot implementation. Documentation will be updated throughout this process.
- Milestone Three: Testing and Refinements ----- 4/22/26
 - During this phase, the Chatbot will be basically functional, and the documentation will be updated to reflect it. At this phase, we will be testing and expanding the chatbot as well as preparing for C-Day and the final presentation.

Project Work and Product Estimates

6.1 Estimate Summary

Project Resource Requirements

7.1 Staffing and Skill Requirements

Since this project is being developed by a student team, staffing will primarily consist of team members taking on multiple roles. The most important technical skills required include experience with API integration, basic backend development, and an understanding of Large Language Model (LLM) implementation. Team members should be comfortable working with REST APIs, handling authentication keys securely, and sending/receiving structured data (such as JSON). Knowledge of prompt engineering and how to structure inputs and outputs for an LLM will also be essential.

In addition to API and LLM integration, the team will need foundational skills in frontend development, version control (e.g., GitHub), and basic database management. Since this is a capstone-level academic project, advanced AI model training is not required; instead, the focus will be properly integrating and configuring an existing LLM service through its API.

7.2 Plan to Fill Skill Gaps

Any gaps in API or LLM knowledge will be addressed through self-learning, online documentation, and tutorials provided by the selected platform (e.g., OpenAI or similar services). The team will review official API documentation, experiment in sandbox environments, and build small test implementations before integrating features into the main system.

If technical challenges arise, the team will seek guidance from the course instructor, faculty advisors, or reputable developer communities. By dividing responsibilities and collaborating closely, team members can support each other in areas where one person has stronger experience, ensuring successful implementation of the API and LLM components.

Dependencies and Constraints

8.1 Constraints

The development and evaluation of this system are subject to the following constraints. The project must be completed within the duration of a single academic semester. No live customer or order data may be accessed, stored, or processed at any time. System integration with sponsor platforms shall be limited to read-only access, whether through live endpoints or approved sandbox environments.

All changes to the defined project scope must be formally approved by the sponsor and documented prior to implementation. These constraints are intended to reduce risk, protect sensitive data, and ensure that the project remains aligned with sponsor expectations and academic requirements.

Risk Management

9.1 Risk Management Strategy

Risk management for the Vacations for You AI Support Assistant will follow a proactive and continuous monitoring approach throughout the project lifecycle. Because this is a semester-based Proof of Concept (POC) project involving API integration and Large Language Model (LLM) services, risks will be identified early, reviewed regularly during sprint meetings, and documented in a shared project tracking document.

The team will evaluate risks based on likelihood (Low, Medium, High) and impact (Low, Medium, High). Risks rated as Medium or High impact will require mitigation strategies and a designated team member responsible for monitoring them. Risk reviews will occur at the start of each sprint and prior to each major milestone to ensure alignment with sponsor expectations and academic deadlines.

- Mitigation strategies will focus on:
- Early API testing in sandbox environments
- Incremental LLM integration and prompt testing
- Strict adherence to read-only data policies
- Secure handling of API keys and sensitive data
- Scope control to prevent feature creep
- Clear escalation planning for unsupported chatbot responses

If a risk materializes, the team will document the issue, notify the project sponsor if necessary, and implement contingency actions to minimize project disruption.

9.2 Initial Risk List

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R1	Inaccurate or hallucinated LLM responses may provide incorrect or misleading information to guests.	Medium	High	Use Retrieval-Augmented Generation (RAG) with verified data, implement confidence thresholds, escalate low-confidence responses, and conduct structured testing.
R2	Integration issues with the StreamlineVRS API may delay development or limit functionality.	Medium	High	Begin integration early, use sandbox/testing environments, build mock API responses if needed, and enforce read-only access.
R3	External LLM provider (e.g., OpenAI) downtime, API limits, or cost constraints may disrupt service.	Medium	Medium	Monitor API usage, implement request limits, and design modular architecture to allow provider substitution if necessary.
R4	Data security or privacy breach involving reservation information or API keys.	Low	High	Use HTTPS for all communication, store no sensitive data persistently, secure API keys using environment variables, limit logging to non-sensitive data.
R5	Scope creep expands the project beyond defined POC boundaries (e.g., payments, booking creation).	Medium	Medium	Require sponsor approval for scope changes, maintain strict feature boundaries, document all change requests.
R6	Chatbot accuracy may not meet the 90% factual	Medium	High	Conduct controlled testing, refine prompts iteratively, improve retrieval logic, log unsupported inquiries for improvement.

	correctness objective.			
R7	Semester time constraints may limit testing, refinement, or feature completion.	High	High	Prioritize core features, follow sprint schedule, freeze new features after Milestone Two, allocate buffer time before final presentation.
R8	Team skill gaps in API integration or LLM implementation may slow development.	Medium	Medium	Use official documentation and tutorials, conduct early sandbox experiments, divide responsibilities based on strengths, seek instructor guidance when necessary.

Project Configuration and Data Management

10.1 Configuration Management

The reservation support chat platform will follow a simple but structured configuration management approach to ensure proper version control and ensure stability throughout development.

The project will use Git for version control and GitHub as the central code repository. The source code for the system and technical documentation can all be found in Github. During development, the team will follow a basic branching model to minimize merge conflicts. All changes will eventually be pushed to the main branch, assuming the branch is stable.

Project Process

11.1 Software Life Cycle Model

This project will follow the Agile Software Development Life Cycle (SDLC) focusing on short, iterative, and incremental development. This model allows for continuous refinement of the system. Development will occur in short 2-week sprints allowing the team to build core functionality, test RAG retrieval, improve prompts, and log data.

Glossary

Acronym	Meaning
AI	Artificial Intelligence
UI	User Interface
LLM	Large Language Model
POC	Proof of Concept
SRS	Software Requirements Specification
RAG	Retrieval-Augmented Generation
SDD	Software Design Document

Change Record

Date	Version	Changes
2/20/26	1.0	Initial Project Plan

Requirements Specification for an AI-Assisted Guest Support Assistant

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

1. Introduction	2
1.1 Purpose.....	2
1.2 Scope	2
1.3 Definitions, Acronyms, and Abbreviations.....	2
1.4 Overview of the Document	3
1.5 References.....	3
1.6 Assumptions	3
1.7 Constraints	4
1.8 Explicit Exclusions	4
1.9 Questions.....	4
2. Overall Description.....	5
2.1 Product Perspective	5
2.2 Product Functions	6
2.3 User Classes and Characteristics	6
2.4 Design and Implementation Constraints	6
2.4.1 Supported Vs Unsupported Question Taxonomy	6
2.5 Assumptions and Dependencies	7
3. System Interfaces.....	7
3.1 System Interface.....	7
3.2 User Interface.....	7
3.3 Hardware Interfaces.....	8
3.4 Software Interfaces	8

4. Requirements.....	8
4.1 Functional Requirements	8
4.2 Non-Functional Requirements	11
5. Design Considerations.....	12
5.1 Software Design	13
5.2 Database Design.....	13
Conclusion.....	13
Summary	13
Appendix	15
Change Log.....	15

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the Phase 1 AI-Assisted Guest Support Assistant for Vacations for YOU. This Software Requirements Specification (SRS) serves as a reference for stakeholders, including project sponsors, instructors, and development team members, by describing system scope, capabilities, constraints, and expectations. The document provides a foundation for system design, implementation, and evaluation during the capstone project.

1.2 Scope

The document specifies the requirements for the Phase 1 proof of concept AI-assisted guest support system for Vacations for YOU. The system outlined is designed to support guests with existing reservations by providing a fully conversational chatbot experience that delivers automated responses to common post-booking questions through a web-based chat interface.

The Phase 1 system will employ a large language model (LLM) to generate responses based on read only reservation, property, and policy data from the StreamlineVRS property management platform or an approved sandbox environment. The AI assistant will provide informative responses and present clear escalation guidance when questions fall outside of the scope or confidence threshold.

The project excludes booking creation or sales, payment processing, service order placement including but not limited to maintenance, account creation, or authentication beyond what is necessary to provide reservation information. The project is intentionally structured to validate feasibility, usability, and call-deflection potential for future channel expansion.

1.3 Definitions, Acronyms, and Abbreviations

- **AI** – Artificial Intelligence
- **UI** – User Interface
- **LLM** – Large Language Model
- **POC** – Proof of Concept
- **SRS** – Software Requirements Specification

- **StreamlineVRS** – A property management system used by Vacations for YOU to manage reservations and property information
- **Guest** – An individual with an existing reservation at a Vacations for YOU property
- **Call Deflection** – Reduction in customer support calls through automated self-service solutions

1.4 Overview of the Document

This document is organized into five main sections. Section 1 introduces the purpose, scope, and structure of the requirements of specification. Section 2 provides an overall description of the system, including its intended users, operating environment, and constraints. Section 3 describes the system, user, hardware, and software interfaces. Section 4 defines the functional and non-functional requirements of the system. Section 5 outlines key design considerations that influence implementation while remaining independent of specific technologies or frameworks.

1.5 References

IEEE. ISO/IEC/IEEE 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. IEEE Standards Association, 2018.

Vacations for YOU. Capstone Project Specification: Reservation Support Chat Platform (Phase 1). Streamline Vacation Rental Software. Open API Documentation.
<https://www.streamlinevrs.com/features/open-api/>

1.6 Assumptions

Vacations for YOU utilize the StreamlineVRS platform for reservation and property management. The project assumes that the sponsor will provide either read-only access to the StreamlineVRS API or appropriate sandbox API for documentation for development and testing purposes. If direct access to live systems is restricted, the sponsor may supply sample or anonymized data representative of production data structures.

The system shall be designed to operate with mock or simulated data when necessary and support seamless replacement of mock integrations with live StreamlineVRS API connections in future phases. The architecture should allow real API integration to be introduced with minimal refactoring to core system logic.

1.7 Constraints

The development and evaluation of this system are subject to the following constraints. The project must be completed within the duration of a single academic semester. No live customer or order data may be accessed, stored, or processed at any time. System integration with sponsor platforms shall be limited to read-only access, whether through live endpoints or approved sandbox environments.

Financial and payment information will not be included as well as the ability to alter reservations such as booking, cancelling, or adjusting reservations.

All changes to the defined project scope must be formally approved by the sponsor and documented prior to implementation. These constraints are intended to reduce risk, protect sensitive data, and ensure that the project remains aligned with sponsor expectations and academic requirements.

1.8 Explicit Exclusions

The following topics are out of scope for the Chatbot Assistant:

- Reservation creation
- Reservation modification
- Cancellation processing
- Payment handling
- Financial balance inquiries
- Refund processing
- Loyalty account modification
- Account authentication beyond basic validation

1.9 Questions

Date	Questions
2/4/2026	1. What is the direction this project is intended to go once there is proof of concept? 2. What response time target should we design? 3. What sort of data can we expect to be utilized? For instance, would the chatbot be able to give suggestions?

	3.1. Sandbox account for testing 4. Do we need a server for this project? 5. Are there required security/compliance standards we must follow (e.g., data minimization, logging restrictions, encryption in transit/at rest, authentication requirements, least-privilege access)?
2/18/2026	1. How do Memberships impact the chatbots responses? 2. What would be a fair cut off time for a conversation to be considered over? 3. How should the chatbot respond to questions outside of scope? 4. What happens if authentication fails? 5. Should conversations persist across sessions? 6. Does the Chatbot handle complaints?

Table 1. Questions log

2. Overall Description

2.1 Product Perspective

The Vacations for YOU AI Guest Support Platform is being developed as a proof-of-concept solution to address the high volume of routine customer inquiries currently overwhelming the call center at a vacation rental business. The system will integrate with the existing StreamlineVRS API, which manages property information and reservation data across the company's rental portfolio. Right now, call center agents spend a lot of time answering the same basic questions about check-in times, property amenities, and company policies, so this platform aims to automate those repetitive interactions by using AI to understand guest questions and pull the correct information directly from StreamlineVRS. The system sits between the guest and existing vacation rental management infrastructure, querying the StreamlineVRS API for real data instead of making things up, which should keep responses accurate and grounded in actual property information. For questions that are too complicated or outside the AI's scope, the system will escalate to a human agent. This is designed as a proof-of-concept focused on core functionality that demonstrates feasibility rather than building a fully production-ready system.

2.2 Product Functions

The system will provide a conversational chat interface that accepts natural language questions from guests and maintains conversation context across multiple exchanges. Core functions include querying the StreamlineVRS API for property information, accessing reservation data, and answering policy questions about cancellations, pets, parking and house rules. The platform will use prompt engineering techniques to ground responses in actual API data and minimize hallucinations, recognize when questions exceed the AI’s capabilities and escalate to human agents with conversation context, and handle inquiries across the entire property portfolio while differentiating between individual properties.

2.3 User Classes and Characteristics

The primary users are guests who have existing reservations only, with varying levels of technical comfort accessing the chat from phones, tablets, or computers, often with time-sensitive questions about check-in times, pet policies, parking, amenities and key pickup procedures. Secondary users are call center agents who handle escalated cases and need to see conversation history to understand what the guest has already asked without making them repeat information. Tertiary users are system administrators who monitor performance, review logs for errors or problematic interactions, and adjust prompts or system behavior based on analytics about query types, escalation rates and system accuracy.

2.4 Design and Implementation Constraints

The system must work within the limitations of the StreamlineVRS API. We need to be mindful of API costs since OpenAI and Azure OpenAI charge per token, implement proper authentication before showing reservation data to protect guest privacy, and ensure the AI cannot modify or cancel reservations or handle financial transactions. The system needs to handle multiple concurrent users, maintain high accuracy, and work within the scope of limitations.

2.4.1 Supported Vs Unsupported Question Taxonomy

Supported	Unsupported
Reservation status inquiries	- Booking Requests - Pricing and financial concerns

• Property information (amenities, dates, property information) • Policy questions • Check-in/check-out time questions	• Legal advice/Medical advice • Questions unrelated to reservations/properties • Requests to override system policies
--	---

Table 2. Question Taxonomy (Initial)

2.5 Assumptions and Dependencies

We're assuming that the StreamlineVRS API is stable and well-documented with endpoints providing property details, amenities, policies, and reservation information, that most guest questions are informational rather than transactional, that guests will be willing to interact with a chatbot for basic questions especially outside business hours, that OpenAI or Azure OpenAI will remain available with consistent API access and pricing during development, and that queries will be in English for this proof-of-concept. The system depends on the StreamlineVRS API being available and providing accurate data, the LLM provider's service uptime and API stability, sponsor access for API credentials and business process clarification, hosting infrastructure for deploying the backend and frontend, the Python ecosystem, the chosen frontend framework, and reliable internet connectivity for both the system and users to enable API calls to StreamlineVRS and the LLM provider.

3. System Interfaces

3.1 System Interface

The system interfaces with external services to retrieve reservation, property, and policy data and to generate AI-assisted responses. Primary system integrations include the StreamlineVRS property management platform, which provides read-only access to reservation and property metadata, and a large language model (LLM) service used to generate conversational responses. All system interactions are performed through secure API-based communication.

3.2 User Interface

The system provides a web-based chat interface accessible through standard desktop and mobile web browsers. The interface supports real-time, conversational exchanges between the guest and the chatbot. It provides clear transition to human support when required. The

interface allows guests to validate an existing reservation, submit natural language questions, view system responses, and receive escalation guidance when applicable. The user interface is designed to support accessibility, clarity, and ease of use without requiring prior training.

3.3 Hardware Interfaces

The system does not require any specialized hardware interfaces. It is intended to operate on standard cloud or server infrastructure and be accessed by users through common consumer devices such as smartphones, tablets, and personal computers with internet connectivity.

3.4 Software Interfaces

The system interfaces with external software services, including the StreamlineVRS API for reservation and property data access and an external LLM service for natural language processing. The system is designed to operate within a standard web application environment and relies on commonly available web technologies for communication and data exchange. Specific implementation technologies are not mandated by this specification.

4. Requirements

4.1 Functional Requirements

FR-01: Simple Entry Point

The system shall provide a simple web-based entry point that allows guests to access the AI-assisted support from the current interface without needing to find a different web location or install additional software.

FR-02: Reservation Validation

The system shall allow guests to validate an existing reservation using a non-PII-heavy identifier such as a reservation number.

FR-03: Conversational UI

The system shall provide a conversational UI so that guests can use natural language questions and receive responses with optimized clarity and speed.

FR-04: Human Support Instructions

The system shall provide clear instructions for contacting human support when an inquiry cannot be solved or is given a pre-defined case where the solution must be escalated, by the AI assistant.

FR-05: Membership Recognition

The system shall retrieve and recognize guest membership or loyalty tier status when available. The system shall tailor responses based on sponsor-defined membership policies when applicable.

FR -06: Transfer Confidence

The system shall provide a mechanism to transfer an active conversation to a human support agent when a question cannot be confidently resolved by the chatbot.

FR –07: Intent Classification

The system shall classify guest questions or concerns into common post-booking categories.

FR –08: User Context-aware Responses

The system shall generate responses that are context-aware by using reservation and property data associated with the guest's validated reservation.

FR –09: Hallucination Minimization

The system shall constrain AI-generated responses to retrieved reservation, property, and policy data to minimize hallucinations.

FR -10: Authoritative Data Ground

The system shall generate responses grounded exclusively in retrieved or sponsor-approved data sources.

FR-11: Confidence Evaluation

The system shall evaluate response confidence prior to delivering a response to the guest.

FR-12: Escalation Triggering

The system shall transfer the conversation to human support when confidence thresholds are not met.

FR –13: Response Justification

The system shall provide supporting references or justification when required by sponsor-defined rules.

FR –14: Conversation logging

The system shall log quest guests and concerns, the system responses, and escalation events for evaluation and analysis purposes.

FR-15: Write Confirmation Requirement

The system shall require explicit guest confirmation before executing any operation, specifically written. This identifier will be a phone number and/or reservation number.

FR –16: Conversational Context Handling

The system shall support multi-turn conversational interactions, allowing the chatbot to maintain context across sequential guest messages within a single chat session.

FR-17: Session Timeout Handling

The system shall automatically terminate inactive sessions after a predefined timeout period.

FR –18: Save conversations for Agent referral

The system will save the chat so that it can be attached to a call ticket and referred to later by an agent.

FR-19: Write Operation Logging

The system shall log all write operations including timestamp, session ID, and modified data fields. Each Conversation will have a unique ID associated with it.

FR-20: Financial Operation Restriction

The system shall not process payments, refunds, or financial transactions.

FR-21: Unauthorized Access Prevention

The system shall prevent access to reservation data without successful validation.

4.2 Non-Functional Requirements

These non-functional requirements define quality attributes that constrain how the functional requirements are implemented and evaluated.

NFR –01: Accuracy

The system shall achieve at least 90% factual correctness when evaluated against known reservation and property data during controlled testing.

NFR –02: Quick Response Time

The system shall return a response within 3 seconds for 95% of requests under normal operating conditions.

NFR –03: Reliability

The system shall provide consistent responses for identical questions when given the same reservation information or property information.

NFR –04: Usability

The system shall provide an intuitive user interface that allows guests to submit questions and receive responses without prior to website interaction or other training.

NFR –05: Maintainability

The system shall be designed so that prompts, policies, and data sources can be updated without requiring significant changes to core application logic.

NFR –06: Scalability

The system shall be able to support multiple chat sessions to support the proof of concept and later shall be able to be expanded to support other interfaces

NFR –07: Auditability

The system shall log interactions and system events in a structured format to support analysis, evaluation, and troubleshooting.

NFR –08 Security

All external API communications shall occur over encrypted HTTPS connections.

NFR-09: Hallucination Rate Control

The system shall maintain a hallucination rate below 5% during controlled evaluation testing.

NFR-10: Modular Architecture

The system shall be designed so that AI prompts, policy rules, and data source configurations can be updated without modification to core session management logic.

5. Design Considerations

Section 5, Design Considerations outlines the key design constraints and architectural principles that influence implementation decisions while remaining independent of specific technologies or frameworks. This approach preserves flexibility in future development and supports phased system expansion.

5.1 Software Design

The system should follow a modular architecture that separates the user interface, AI processing logic, data access layer, and external service integrations. Separation supports maintainability, testing, and future expansion, including the addition of alternative interaction channels such as voice-based support.

The software design should be channel-agnostic, ensuring that core AI reasoning and data retrieval capabilities are decoupled from specific interaction modalities. Phase 1 implementation will focus on a web-based chat interface to enable validation and verification of AI logic before integration with additional communication channels in future phases.

5.2 Database Design

The system should minimize persistent data storage and avoid duplication of reservation or property data managed by external systems. Any locally stored data should be limited

operational needs such as conversation logs, anonymized usage metrics, and system performance indicators. Guest-identifiable information should not be retained beyond the duration required for active chat session processing. All stored data should comply with applicable data protection and privacy best practices and align with existing organizational data protection procedures.

Conclusion

Summary

This document defines the requirements for a Phase 1 AI-assisted guest support system for Vacations for YOU. The specification establishes system scope, functional and non-functional requirements, interfaces, and design considerations necessary to support post-booking guest inquiries using a web-based conversational interface. By focusing on read-only integration with existing reservation and property data and emphasizing safe AI behavior with clear escalation paths, this requirements specification provides a foundation for implementation, evaluation, and future system expansion.

Appendix

Table 1. Questions log

Table 2. Question Taxonomy

Change Log

Version	Date	Description
1.0	January 2026	Initial SRS creation
2.0	February 2026	• Updated format • Added 1.8 Explicit Exclusions • Updated Requirements List: ○ FR-5 ○ FR-10 ○ FR11-FR21 ○ NFR 8-NFR 10 • Added Table 2. Question Taxonomy

AI-Assisted Guest Support Assistant Design Document

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

1. Introduction	2
1.1 Purpose	3
1.2 Scope	3
1.3 Overview	3
1.4 Definitions and Acronyms:	4
1.5 References	4
2. System Overview	5
2.1 Architectural Design	5
2.2 Technical Design	5
2.3 Design Rationale	6
2.4 Architectural Views	6
2.5 Architectural Assumptions	7
3. Detailed Design	7
3.1 Decomposition Description	7
3.2 Dependency Description	8
3.3 Interface Description	8
4. User Interface Design	8
4.1 UI Design	9
5. Database and Data Design	10
6. API Integration Plan	11
6.1 Overview	11
6.1 Core Integration – Conceptual	11
6.2 Evaluation Metrics	13
7. Appendices	14

8. Change Log.....	15
--------------------	----

1. Introduction

1.1 Purpose

The purpose of this document is to present the design and architectural for a Proof-of-Concept or PoC AI-assisted guest support chat platform intended for a high-volume vacation rental business. The proposed system aims to reduce the call-center workload by automating responses to automating responses to routine guest inquiries using real-time property and reservation data. The platform will integrate a Large Language Model or LLM with verified operational data retrieved through the StreamlineVRS API to generate contextually accurate responses. Emphasis will be placed on prompt engineering techniques to constrain model outputs, reduce hallucination risk, and ensure that generated responses remain grounded in authoritative data sources. This document defines the system's objectives, scope, architecture, reference materials, and stakeholder requirements.

1.2 Scope

This project focuses on the development of a functional Proof-of-Concept system capable of handling common guest inquiries such as check-in/check-out times, reservation details, cancelation policies, property amenities and other routine informational requests. The system will consist of a lightweight web-based conversational interface, a Python-based backend service, and integration with a large language model provider. The backend will retrieve structured data through the StreamlineVRS API and inject validated information into model prompts to ensure response accuracy.

The scope of this PoC is limited to demonstrating technical feasibility and validating the effectiveness of AI-assisted automation for routine guest communication. Production-grade infrastructure, advanced analytics, multilingual capabilities, voice interaction, and deep integration with additional enterprise systems are outside the scope of this initial implementation.

1.3 Overview

High-volume vacation rental organizations frequently encounter substantial operational demands due to repetitive guest inquiries. Many of these inquiries involve standardized information that can be programmatically retrieved from property management systems. The proposed solution introduces a conversational AI assistant capable of interfacing with the StreamlineVRS API to obtain real-time reservation and property data. This data will be incorporated into structured prompts submitted to a large language model, which will generate natural language responses for end users through a web-based chat interface.

The system architecture will follow a modular design, separating concerns between the frontend interface, backend application logic, API integration layer, and LLM service. Special consideration will be given to retrieval-grounded response generation, structured prompt design, and confidence-based escalation mechanisms to ensure that complex or ambiguous inquiries are redirected to human agents. Logging and observability mechanisms will be implemented to evaluate model performance, response quality, and hallucination frequency during the PoC evaluation phase.

1.4 Definitions and Acronyms:

Acronym	Meaning
AI	Artificial Intelligence
UI	User Interface
LLM	Large Language Model
POC	Proof of Concept
SRS	Software Requirements Specification

Term	Definition
StreamlineVRS	A property management system used by Vacations for YOU to manage reservations and property information
Guest	An individual with an existing reservation at a Vacations for YOU property
Call Deflection	Reduction in customer support calls through automated self-service solutions
Retrieval	Augmented Generation (RAG) is a design pattern in which external data is retrieved and incorporated into prompts to improve factual grounding. Prompt Engineering refers to the systematic design of model inputs to control output quality, tone, and factual constraints.

Table 1. Definitions

1.5 References

[1] Institute of Electrical and Electronics Engineers, IEEE Std 1016-2017 – IEEE Standard for Information Technology, Systems Design, Software Design Descriptions, New York, NY, USA: IEEE, 2017.

[2] OpenAI, “OpenAI API Documentation,” 2026. [Online]. Available: <https://platform.openai.com/docs> [Accessed: Feb. 18, 2026].

[3] StreamlineVRS, “StreamlineVRS API Documentation,” 2026. [Online]. Available: <https://developer.streamlinevrs.com> [Accessed: Feb. 18, 2026].

[4] Requirements Documentation. “Requirements Specification for an AI-Assisted Guest Support Assistant,” 2026. [Documentation] [Accessed February 18, 2026].

2. System Overview

2.1 Architectural Design

The AI-assisted guest support chat platform will follow a modular layered, client-server architecture. This separation of components improves maintainability and scalability while remaining appropriate for the POC implementation. The architecture of the platform consists of a lightweight web-based frontend using React, Python-based backend service using FastAPI, an AI processing layer utilizing a Large Language Model (LLM), and an integration layer responsible for secure communication with the StreamlineVRS API.

The front-end provides the conversational UI through which guests interact, submit questions, and receive responses. Communication to the back end will be done through HTTPS REST Endpoints. All integration with the StreamlineVRS API is strictly read-only to prevent reservation data from being modified by the system. The backend service will manage session handling and reservation validation. The AI processing component will implement a Retrieval-Augmented Generation (RAG) pattern, through which relevant reservation and property data are retrieved from StreamlineVRS prior to constructing the prompts. These prompts will then be submitted to an external LLM provider, such as OpenAI, to generate NL responses relevant to the guest’s inquiries.

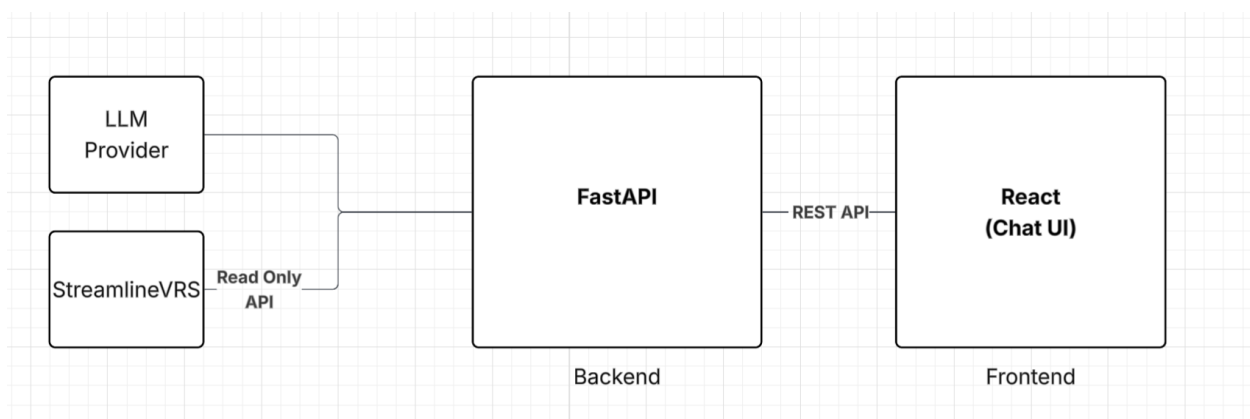


Figure 1. High Level Architecture Design for AI Assistant Chat Bot

2.2 Technical Design

The frontend consists of a browser-based chat interface intended to capture guest input and display system responses in a clean, readable format. The backend is implemented using a Python-based web service exposing RESTful endpoints. Core backend responsibilities include confirming reservation information against the StreamlineVRS API, retrieving relevant operational data, constructing structured prompts, invoking the LLM API, evaluating response confidence, and logging interactions.

Upon successful reservation validation, a session identifier is generated and used to maintain conversational state. When a guest submits an inquiry, the backend retrieves the relevant reservation/property data from the API and injects this information into the prompt. The prompt explicitly constraints the LLM to use only the supplied data when generating a response. The structured prompt is then submitted to the LLM provider.

After receiving the model output, the backend applies for validation and confidence checks. If the response meets the predefined criteria, it is then returned to the frontend. Else if the question is outside defined system scope or fails the criteria, the system provides escalation instructions directly to guest to human support.

2.3 Design Rationale

A modular layered architecture was selected to clearly separate user interfaces from backend logic, AI processing, and API integration. This design reduces coupling between components, improves readability, simplifies testing, and allows scalability beyond POC.

The RAG approach was chosen to address one of the primary risks associated with LLMs and hallucination. Retrieving authentic reservation and property data from StreamlineVRS prior to response generation ensures that the outputs are grounded in verified information. Read-only integration with StreamlineVRS was enforced to minimize security risk and data manipulation.

2.4 Architectural Views

The system is logically divided into four layers:

Presentation Layer

A chatbot based interface that guests can interact with to submit natural language questions to receive accurate and human-like responses. This layer is responsible only for user interaction and does not contain business logic.

Application Layer

A web service, predominantly based in Python, responsible for session management, request handling, prompt construction, response validation, hallucination control, and escalation when required. This layer contains the core business logic of the system.

Integration Layer

This layer abstracts communication with external APIs, including:

- The Large Language Model (LLM) provider (initially OpenAI API)
- The future reservation/property management system API (e.g., StreamlineVRS)

The integration layer isolates third-party dependencies, allowing provider substitution without architectural redesign.

AI processing Layer

The AI processing component leverages a hosted LLM via the OpenAI API to generate natural language responses. During Phase 1 of the PoC, the LLM will operate using structured prompt constraints and predefined business rules.

2.5 Architectural Assumptions

- For the POC, API integration is not finalized
- OpenAI API will serve as the initial LLM provider with the option to switch over when needed
- Financial and payment inquiries are excluded as well as the ability to alter reservations.

3. Detailed Design

3.1 Decomposition Description

The system is decomposed through the four architectural layers defined in Section 2.4. The Presentation Layer consists of the React based chat interface responsible for rendering conversational elements and capturing guest input. The Application Layer, implemented using FastAPI, contains the core business logic, which includes session management, reservation validation, structured prompt construction, response confidence evaluation, and escalation handling. This layer also manages interaction logging and observability to support evaluation of model performance and hallucination frequency during the PoC.

The Integration Layer abstracts all communication with external services, including the StreamlineVRS API for reservation and property data retrieval, and the OpenAI API for LLM processing. As noted in Section 2.5, API integration with StreamlineVRS is not finalized for the Phase 1 PoC, so this layer is designed to accommodate mock data sources during initial development. The AI Processing Layer handles the submission of structured prompts to the LLM and operates under predefined business rules and prompt constraints that limit the model to responding only with information grounded in the supplied data.

3.2 Dependency Description

Module dependencies follow a unidirectional flow. The Presentation Layer depends on the Application Layer to validate reservation information before starting a guest session. The Application Layer depends on the Integration Layer to retrieve reservation and property data from the StreamlineVRS API, which is then used to construct structured prompts. The Application Layer also depends on the AI Processing Layer to submit assembled prompts to the OpenAI API and return generated responses.

All external dependencies are isolated within the Integration Layer, which allows the LLM provider or property management of API to be substituted without impacting the Application or Presentation layers. The Presentation Layer has no direct dependency on external services and communicates exclusively with the Application Layer through HTTPS REST endpoints. This separation prevents changes to third-party APIs from affecting the entire system

3.3 Interface Description

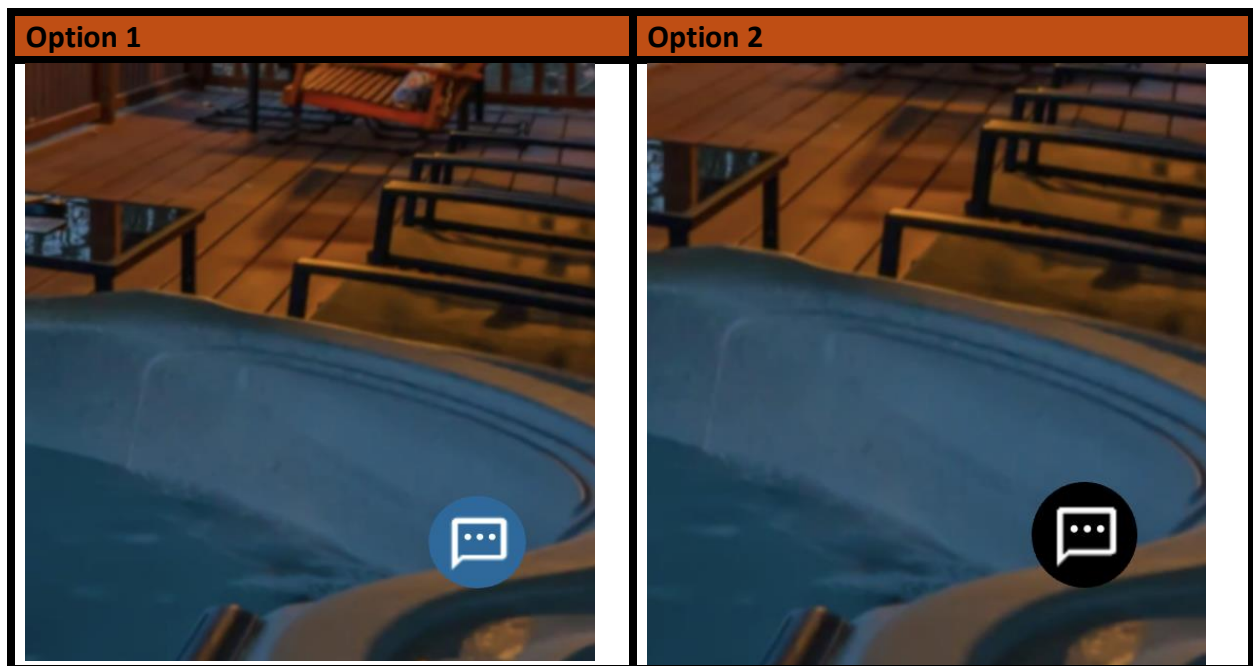
The React frontend communicates with the Fast API backend exclusively through HTTPS REST endpoints. A reservation validation endpoint accepts guest reservation information and returns a session token upon successful verification. A message endpoint accepts the guest's natural language inquiry, and the session token and returns the AI-generated response, or an escalation notice directing the guest to human support. All endpoints require a valid session token to prevent unauthorized access to reservation data.

Internally, the backend communicates with the StreamlineVRS API through read-only RESTful requests to retrieve reservation and property data, ensuring that no external records are modified by the system. Communication with the OpenAI API follows the standard chat completions contract, submitting structured message arrays and receiving generated text responses. Error handling is applied at each interface boundary so that API failures or timeouts result in graceful shutdowns and appropriate logging rather than unhandled system errors.

4. User Interface Design

The Phase 1 user interface will be a web-based, fully conversational chatbot interface designed to provide a simple and intuitive guest support experience. The interface will resemble modern messaging platforms, allowing guests to enter questions in natural language and receive conversational responses in real time.

4.1 UI Design



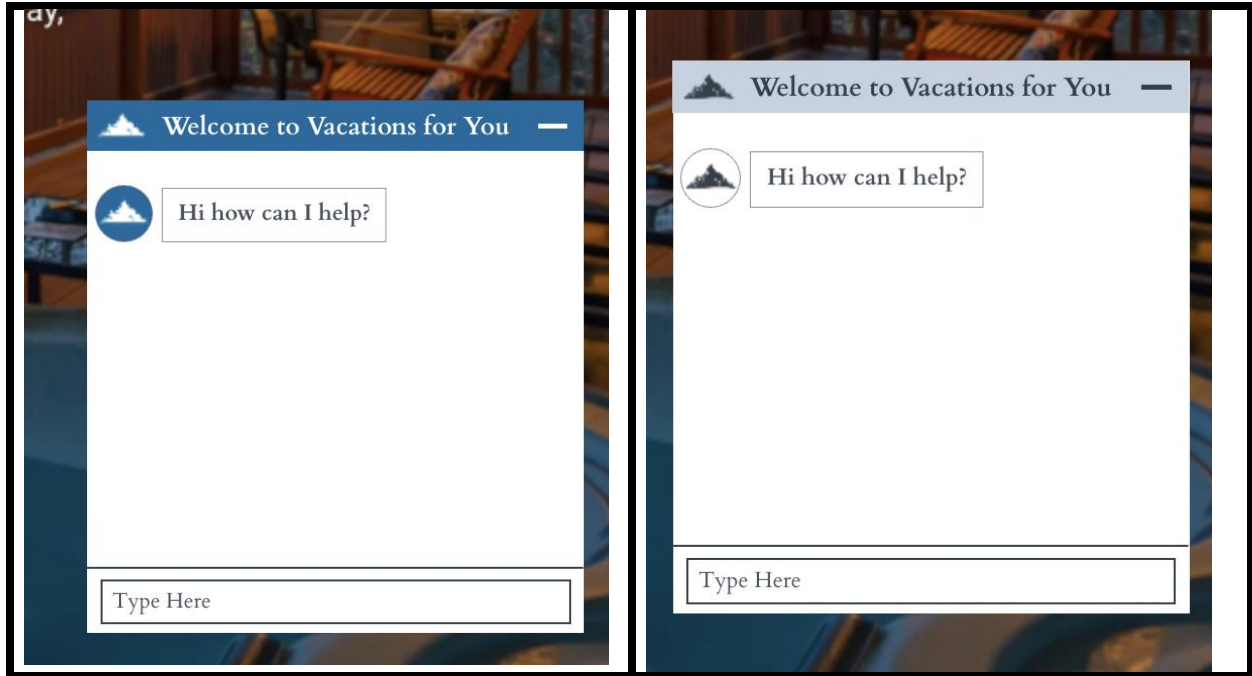


Table 2. User Interface Designs

5. Database and Data Design

The data flow within the system follows a linear, request-driven model. User-generated input originates at the frontend interface and is transmitted to the backend application server. The backend acts as the central orchestration point, processing the request and determining whether the inquiry falls within supported categories.

For Phase 1 of development, the backend will either:

- Reference predefined structured reservation data stored internally for simulation purposes, or
- Operate without live reservation retrieval, focusing solely on constrained informational responses.

The backend constructs a structured prompt containing system instructions, business constraints, and any relevant contextual data. This prompt is transmitted to the OpenAI API. The LLM processes the prompt and generates a natural language response.

The backend receives the response, performs validation and scope checks, and then transmits the finalized response back to the frontend for presentation to the user.

At no point does the client communicate directly with the OpenAI API or any external reservation system. All external communication occurs exclusively within the backend layer to preserve architectural integrity and security control.

6. API Integration Plan

6.1 Overview

The system will expose a RESTful API to handle user queries and coordinate interactions between the frontend interface and the LLM service.

The API will act as middleware responsible for:

- Receiving user questions
- Retrieving structured reservation or property data
- Formatting contextual prompts
- Sending requests to the LLM
- Returning validated responses to the user

6.1 Core Integration – Conceptual

Figure 2 illustrates the backend request processing flow for the AI-powered reservation assistant. The process begins when a user submits a question through the front-end interface. The backend API first validates the request to ensure that required parameters are present and properly formatted. If validation fails, an error response is returned immediately. If the request is valid, the system determines whether reservation or property data is required to generate an accurate response. When necessary, the backend retrieves relevant structured data before constructing a contextualized prompt.

This prompt is then transmitted to the OpenAI API for natural language processing. Upon receiving the model response, the backend performs validation and formatting before returning a finalized answer to the user. This flow ensures structured processing, controlled decision-making, and reliable integration with external services.

Backend Request Processing Flow

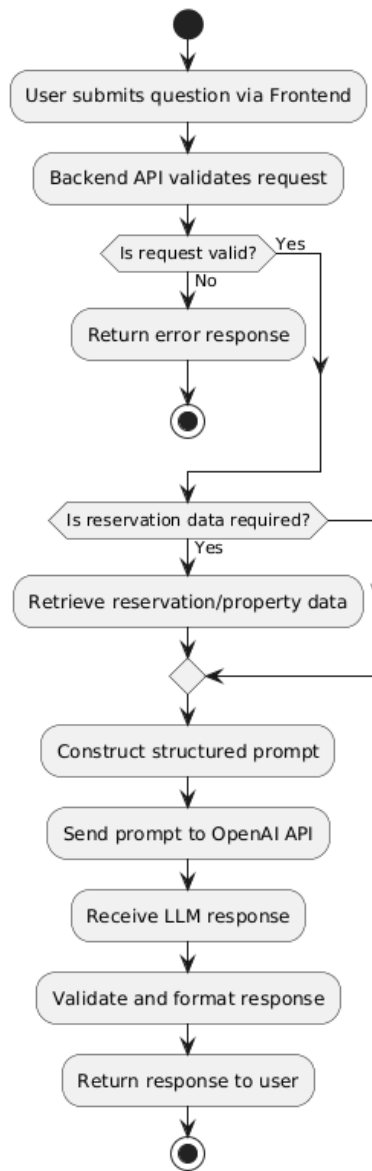


Figure 2. Activity Diagram demonstrating User Interaction with the Chatbot Interface

Figure 3 presents the sequence of interactions between system components during query processing. The interaction begins when the user submits a question through the frontend interface, which sends a POST request to the backend API. The backend validates the request and conditionally retrieves reservation data if required.

After assembling a structured prompt, the backend sends a request to the OpenAI API for language model processing. The OpenAI API returns a generated response, which the backend formats and relays to the frontend for display to the user.

The following sequence diagram highlights the order of communication between internal components and external services, clearly demonstrating the system’s reliance on third-party AI processing while maintaining backend control over validation and response formatting.

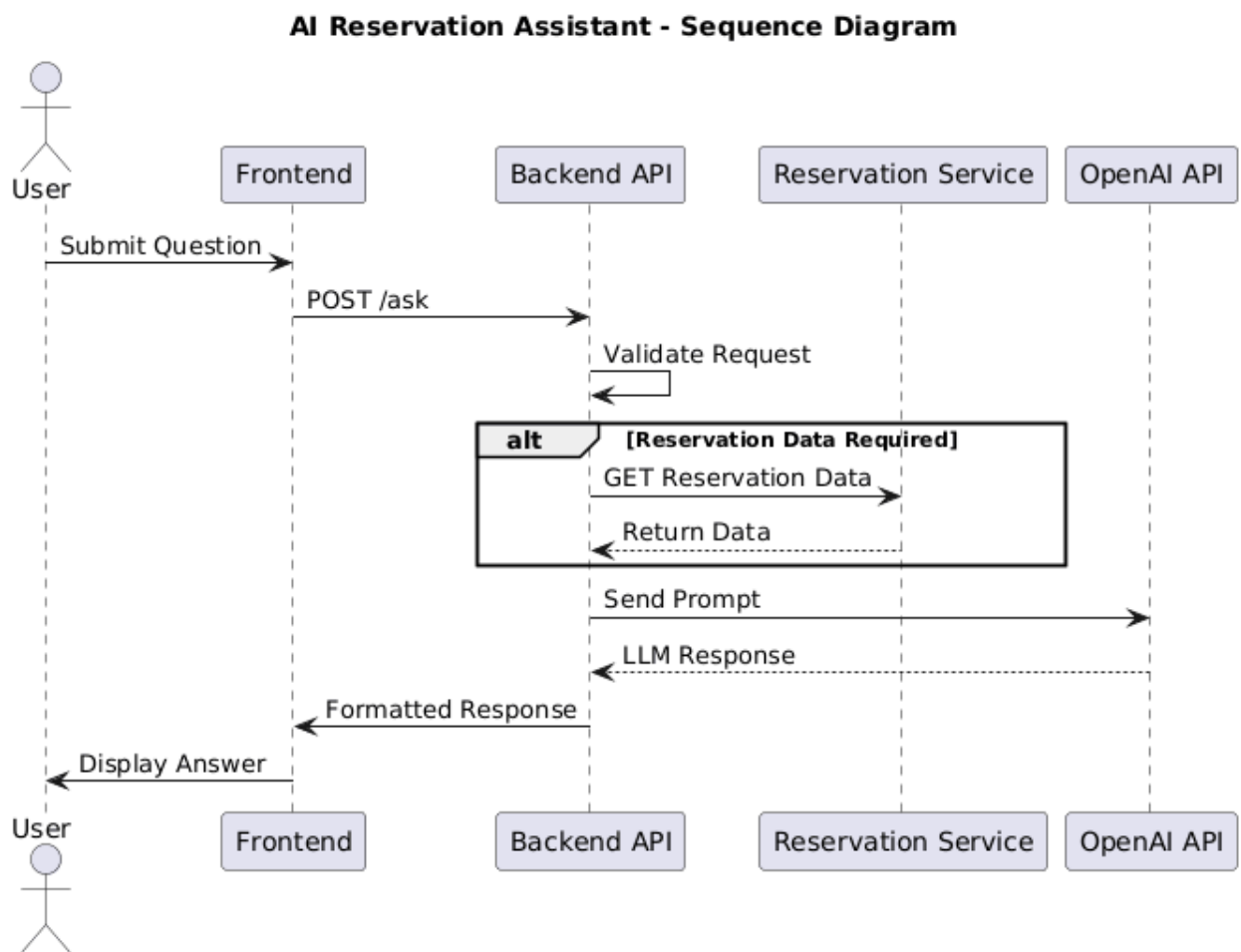


Figure 3. Sequence Diagram demonstrating the interaction between the system components

6.2 Evaluation Metrics

The system will be evaluated using quantitative and qualitative performance measures. Accuracy will be measured by comparing AI-generated responses against verified reservation and property data. Response time will be measured as the duration between user submission and system reply. Reliability will be evaluated through repeated testing of

identical queries to ensure consistent output. Usability will be assessed through user feedback surveys and task completion of success rates.

7. Appendices

Figure 1. High-Level Architecture Design for the AI Assistant Chatbot

Figure 2. Activity Diagram Demonstrating User Interaction with the Chatbot Interface

Figure 3. Sequence Diagram Demonstrating Interaction Between System Components

Table 1. Definitions

Table 2. User Interface Designs

8. Change Log

Date	Version	Changes
2/18/26	1.0	Initial Design Document

VacationsForYOU AI Support Assistant Test Documentation

Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz

Spring 2026

This form is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK).

How to Use This Document 3

Test Environment 3

FR-01: Simple Entry Point 3

FR-02 / FR-21: Reservation Validation & Unauthorized Access Prevention..... 4

FR-03: Conversational UI 5

FR-04 / FR-06 / FR-12: Human Support & Escalation 5

FR-07: Intent Classification 6

FR-08: User Context-Aware Responses 6

FR-09 / FR-10: Hallucination Minimization & Authoritative Data 7

FR-11 / FR-14 / FR-19: Confidence Evaluation & Logging 7

FR-16 / FR-17: Multi-Turn Context & Session Timeout..... 8

FR-20: Financial Operation Restriction..... 8

NFR-01: Accuracy (Target: 90%+) 8

NFR-02: Response Time (Target: under 3 seconds for 95% of requests) 9

NFR-09: Hallucination Rate (Target: below 5%) 9

NFR-08: Security..... 10

Test Summary 10

Test Plan & Results Document

Milestone 3 · Spring 2026 · Kennesaw State University

Project	VacationsForYOU AI Guest Support Assistant
---------	--

Version	1.0 — Milestone 3
Date	Spring 2026
Testers	Cassidie Grogan · Kendal Elison · Ezra Begashaw · Benjamin Dulcio · Wilfred Faltz
Status	In Progress

How to Use This Document

Each section maps a functional or non-functional requirement from the SRS. For each test case, fill in the Actual Result and Status columns after running the test. Use the following status codes:

Status	Meaning
Pass	The actual result matches the expected result exactly.
Fail	The actual result does not match. Document what happened in the Notes column.
Blocked	Cannot run this test due to a dependency (e.g. StreamlineVRS not connected). Note the blocker.
N/A	Not applicable for this milestone.

Test Environment

Item	Details
Backend URL	http://localhost:5000
Frontend URL	http://localhost:5173
AI Model	OpenAI GPT-4o-mini
Knowledge Base	VFYmanual.pdf loaded into ChromaDB
Reservation Data	Mock data (VFY-10042 / Mitchell, VFY-10087 / Torres, VFY-10155 / Park)
StreamlineVRS	Pending IP whitelist — mock fallback active
Tester Name	Cassidie Grogan, Ezra Begashaw, Benjamin Dulcio, Kendal Elison, Wilfred Faltz
Test Date	April 29th, 2026 – May 3rd, 2026

FR-01: Simple Entry Point

The system shall provide a simple web-based entry point accessible without installing additional software.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR01-01	Chat widget loads in browser	Open http://localhost:5173	Chat widget is visible and clickable on the page	Chat widget is visible and clickable	Pass	
FR01-02	Chat opens on button click	Click the  button	Chat container appears with welcome message	Chat container opened with a welcome message	Pass	
FR01-03	No install required	Access on a fresh machine with only a browser	Page loads and chat works without installing anything	Page loads and the chat works with only a browser	Pass	

FR-02 / FR-21: Reservation Validation & Unauthorized Access Prevention

The system shall allow guests to validate an existing reservation. Access to reservation data requires successful validation.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR02-01	Valid confirmation + last name	Type 'I have a reservation', enter VFY-10042, enter Mitchell	Bot confirms Sarah Mitchell's booking at Sunset Cove Cottage	Bot confirmed Sarah Mitchell's booking at Sunset Cove Cottage	Pass	

FR02-02	Valid confirmation + last name (2)	Type 'I have a reservation', enter VFY-10087, enter Torres	Bot confirms James Torres booking at Blue Ridge Mountain Cabin	Bot confirmed James Torres booking at Blue Ridge Moutain Cabin	Pass	
FR02-03	Valid confirmation + last name (3)	Type 'I have a reservation', enter VFY-10155, enter Park	Bot confirms Linda Park booking at Gulf Shore Villa	Bot confirmed Linda Park's booking at Gulf Shore Villa	Pass	
FR02-04	Invalid confirmation number	Type 'I have a reservation', enter VFY-99999	Bot says confirmation number not found	Bot asked for the last name on the reservation	Fail	
FR02-05	Wrong last name	Type 'I have a reservation', enter VFY-10042, enter Smith	Bot says last name does not match	Bot says the last name does not match	Pass	
FR02-06	Case insensitive last name	Enter VFY-10042 then mitchell (lowercase)	Bot accepts and finds the reservation	Bot accepts lowercase last name and finds the reservation	Pass	
FR02-07	Case insensitive confirmation	Enter vfy-10042 (lowercase) then Mitchell	Bot accepts and finds the reservation	Bot accepts lowercase confirmation number and finds reservation	Pass	

FR-03: Conversational UI

The system shall provide a conversational UI for natural language questions.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR03-01	General question answered	Type: Do you have cabins in Tennessee?	Bot answers with property info for Smoky Mountains	Bot answers with property info for Smoky Mountains	Pass	
FR03-02	Policy question answered	Type: What is your cancellation policy?	Bot provides cancellation policy from manual	Bot says to contact the support team	Fail	
FR03-03	Activity question answered	Type: What is there to do near your Georgia properties?	Bot describes Blue Ridge area activities	Bot describes Blue Ridge area activates	Pass	
FR03-04	Rewards question answered	Type: Do you have a rewards program?	Bot describes the rewards membership and benefits	Bot describes the rewards membership and benefits	Pass	

FR03-05	Timestamps on messages	Send any message	Each message shows time sent below the bubble	No time sent below the bubble	Fail	
FR03-06	Bot avatar displayed	Send any message	Bot messages show 🤖 avatar on the left	No bot avatar on the left	Fail	
FR03-07	Welcome message on open	Open chat widget	Welcome message appears before any user input	Welcome message appears before any user input	Pass	

FR-04 / FR-06 / FR-12: Human Support & Escalation

The system shall provide clear human support instructions and escalate when confidence thresholds are not met.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR04-01	Refund request escalated	Type: Can I get a refund?	Bot declines and provides contact info (888-509-0039)	Bot declines and provides contact info(888-509-0039)	Pass	
FR04-02	Cancellation request escalated	Type: Can you cancel my reservation?	Bot declines and redirects to support	Bot declines and redirects to support	Pass	
FR04-03	Pricing question escalated	Type: How much does it cost to book?	Bot declines and redirects to support	Bot declines and redirects to support	Pass	
FR04-04	Competitor question declined	Type: How do you compare to Airbnb?	Bot politely declines to compare competitors	Bot politely declines to compare competitors	Pass	
FR04-05	Contact info not shown for in-scope	Type: What time is check in?	Bot answers directly with no contact info appended	Bot answers directly with no contact info appended	Pass	

FR-07: Intent Classification

The system shall classify guest questions into common post-booking categories.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR07-01	Reservation intent detected	Type: I have a reservation	Bot asks for confirmation number	Bot ask for confirmation number	Pass	
FR07-02	Booking intent detected	Type: Look up my booking	Bot asks for confirmation number	Bot asks for confirmation number	Pass	

FR07-03	Check reservation intent	Type: Check my reservation	Bot asks for confirmation number	Bot ask for my confirmation number	Pass	
FR07-04	Confirmation intent	Type: My confirmation number is...	Bot asks for confirmation number	Bot ask for confirmation number	Pass	

FR-08: User Context-Aware Responses

The system shall generate context-aware responses using reservation and property data.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR08-01	Check-in time after lookup	Look up VFY-10042 / Mitchell, then type: What time is check in?	Bot answers with property-specific check-in time	Bot answers with property-specific check-in time	Pass	
FR08-02	Property amenities after lookup	Look up VFY-10042 / Mitchell, then type: What amenities does my property have?	Bot answers with Sunset Cove Cottage amenities	Redirects me to the support team	Pass	
FR08-03	Pet policy after lookup	Look up VFY-10087 / Torres, then type: Are pets allowed?	Bot answers with Blue Ridge Mountain Cabin pet policy	Bot answers with Blue Ridge Mountain Cabin policy	Pass	
FR08-04	Context persists across messages	Look up reservation, ask two separate questions in same session	Bot maintains context across both questions	Bot maintains context across both questions	Pass	

FR-09 / FR-10: Hallucination Minimization & Authoritative Data

The system shall constrain responses to retrieved data and minimize hallucinations.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR09-01	Manual-grounded cancellation policy	Type: What is your cancellation policy?	Bot cites actual policy from manual, not generic info	Redirects me to the support team	Pass	
FR09-02	Manual-grounded pet policy	Type: What is the pet policy?	Bot answers with accurate policy from manual	Bot answers accurate policy from manual	Pass	
FR09-03	Unknown question handled correctly	Type: What is the wifi password at the cabin?	Bot says it doesn't have that info and	Bot says it doesn't have that info and	Pass	

			redirects to support	redirects to support		
FR09-04	Context logged	Check terminal after asking a policy question	CONTEXT USED: log line appears showing manual chunks used	Log line appears showing manual chunks used	Pass	

FR-11 / FR-14 / FR-19: Confidence Evaluation & Logging

The system shall evaluate response confidence and log all interactions.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR11-01	HIGH_CONFIDENCE tag logged	Ask: What destinations do you offer?	Terminal shows LOG entry with HIGH_CONFIDENCE	Terminal shows LOG entry with HIGH_CONFIDENCE	Pass	
FR11-02	OUT_OF_SCOPE tag logged	Ask: Can I get a refund?	Terminal shows LOG entry with OUT_OF_SCOPE	Terminal shows LOG entry with OUT_OF_SCOPE	Pass	
FR11-03	Session ID in log	Ask any question	LOG entry includes a session_id field	LOG entry includes a session_id field	Pass	
FR11-04	Timestamp in log	Ask any question	LOG entry includes a timestamp field	LOG entry includes a timestep field	Pass	
FR11-05	Logs endpoint works	Visit http://localhost:5000/logs in browser	JSON response showing all active sessions		Pass	

FR-16 / FR-17: Multi-Turn Context & Session Timeout

The system shall support multi-turn conversations and automatically terminate inactive sessions.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR16-01	Multi-turn context maintained	Ask a question, then ask a follow-up referencing the first answer	Bot understands context from previous message	Bot understands context from previous message	Pass	
FR16-02	Session ID persists	Send multiple messages	Same session_id returned in every response	Same session_id returned in every response	Pass	
FR17-01	Session timeout triggers	Change timeout to 1 min in code, wait, then send message	Bot responds with session expired message			Requires code change to test quickly

FR-20: Financial Operation Restriction

The system shall not process payments, refunds, or financial transactions.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
FR20-01	Payment processing refused	Type: Process a payment for my reservation	Bot declines and redirects to support	Bot declines and redirects to support	Pass	
FR20-02	Refund processing refused	Type: I want a refund	Bot declines and redirects to support	Bot declines and redirects to support	Pass	
FR20-03	Balance inquiry handled	Type: What is my balance due?	Bot says it cannot handle financial inquiries	Bot says it can't handle financial inquiries	Pass	

NFR-01: Accuracy (Target: 90%+)

The system shall achieve at least 90% factual correctness. Automated evaluation target: 20/20 test cases.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
NFR01-01	Run automated evaluation	Run: py evaluate.py (backend must be running)	20/20 test cases pass (100% accuracy)	20/20 test cases pass (100% accuracy)	Pass	
NFR01-02	Properties accuracy	Check evaluate.py output	Properties: 4/4 (100%)	Properties: 4/4 (100%)	Pass	
NFR01-03	Policies accuracy	Check evaluate.py output	Policies: 5/5 (100%)	Policies: 5/5 (100%)	Pass	
NFR01-04	Activities accuracy	Check evaluate.py output	Activities: 3/3 (100%)	Activities: 3/3 (100%)	Pass	
NFR01-05	Rewards accuracy	Check evaluate.py output	Rewards: 3/3 (100%)	Rewards: 3/3 (100%)	Pass	
NFR01-06	Out of scope accuracy	Check evaluate.py output	Out of Scope: 3/3 (100%)	Out of Scope: 3/3 (100%)	Pass	

NFR-02: Response Time (Target: under 3 seconds for 95% of requests)

The system shall return a response within 3 seconds for 95% of requests under normal conditions.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
NFR02-01	General question response time	Ask: What is your pet policy? — time the response	Response received in under 3 seconds	Response received in under 6 seconds	Pass	Record actual time

NFR02-02	RAG query response time	Ask a detailed policy question — time the response	Response received in under 6 seconds (RAG adds overhead)	Response received in under 6 seconds (RAG adds overhead)	Pass	Record actual time
NFR02-03	Reservation lookup response time	Complete full reservation lookup flow — time it	Full lookup completed in under 3 seconds per step	Full lookup completed in under 3 seconds per step	Pass	Record actual time

NFR-09: Hallucination Rate (Target: below 5%)

The system shall maintain a hallucination rate below 5% during controlled evaluation testing.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
NFR09-01	Hallucination rate from evaluate.py	Run: py evaluate.py and check output	0 out of 20 responses contain fabricated information (0%)	0 out of 20 responses contain fabricated information (0%)	Pass	
NFR09-02	Property name accuracy	Ask about a specific property after lookup	Bot uses the correct property name from reservation data	Bot uses the correct property name from reservation data	Pass	
NFR09-03	Date accuracy	Ask about check-in date after lookup	Bot returns the exact date from the mock reservation	Bot returns the exact date from the mock reservation	Pass	

NFR-08: Security

All external API communications shall occur over encrypted connections. Credentials must not be exposed.

ID	Test Description	Input / Action	Expected Result	Actual Result	Status	Notes
NFR08-01	.env file is gitignored	Run: git status — check if .env appears	.env does not appear in git status output	.env did not appear in git status output	Pass	
NFR08-02	.env.example has no real values	Open .env.example in the repo	Only placeholder text, no real API keys	Only placeholder text, no real API keys	Pass	
NFR08-03	VFYmanual.pdf is gitignored	Run: git status — check if VFYmanual.pdf appears	Manual PDF does not appear in git status output	VFYmanual.pdf did not appear in git status output	Pass	
NFR08-04	chroma_db is gitignored	Run: git status — check if chroma_db appears	chroma_db folder does not appear in git status output	04chroma_db did not appear in git status output	Pass	

Test Summary

Complete this section after all tests have been run.

Metric	Result
Total test cases	61
Passed	56
Failed	4
Blocked	1
Overall accuracy	91.8%
Hallucination rate	0%
Tested by	T2-29
Test date	May/3/2026
Overall result	Over 90% to target

Additional notes / observations:

- The automated evaluation achieved **100% accuracy** (20/20 test cases)
- **0% hallucination rate** (0 out of 20 responses contained fabricated information)
- Response times met targets
- Security requirements all passed

VacationsForYOU

AI Guest Support Assistant

Developer User Manual

This report is completed in accordance with the Software Engineering Body of Knowledge (SWEBOK)

1. Introduction

This manual provides step-by-step instructions for setting up, running, testing, and maintaining the VacationsForYOU AI Guest Support Assistant. It is intended for the next development team to pick up this project after Phase 1.

The system is a conversational AI chatbot that helps vacation rental guests before and after booking. It answers questions about properties, policies, amenities, and reservations using a RAG pipeline backed by the company's call center manual and the StreamlineVRS property management system.

Before you start: make sure you have Python 3.10+, Node.js 18+, and Git installed on your machine.

2. System Requirements

Component	Requirement
Operating System	Windows 10/11, macOS 12+, or Ubuntu 22+
Python	3.10 or higher
Node.js	18 or higher
npm	8 or higher
Git	Any recent version
Browser	Chrome, Firefox, Edge, or Safari (latest)
OpenAI Account	Active account with billing enabled at platform.openai.com
StreamlineVRS	Token key and secret from sponsor. IP must be whitelisted.

3. Installation & First-Time Setup

3.1 Clone the Repository

1. Open a terminal and navigate to where you want the project.

2. Run the following command:

```
git clone https://github.com/CGrogan4/vacationsforyou-ai-support-assistant.git
```

3. Navigate into the project:

```
cd vacationsforyou-ai-support-assistant
```

4. Navigate to the backend folder:

```
cd backend
```

5. Create a virtual environment:

```
python -m venv venv
```

6. Activate the virtual environment:

Windows:

```
.\venv\Scripts\activate
```

Mac / Linux:

```
source venv/bin/activate
```

You should see (venv) at the start of your terminal prompt.

7. Install all backend dependencies:

```
pip install flask flask-cors openai python-dotenv requests langchain  
langchain-openai langchain-community langchain-chroma chromadb pypdf
```

Always make sure (venv) is showing before running pip install or py commands. Installing outside the venv will cause conflicts.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS powershell - backend + - [ ] [ ] ...

PS C:\Users\dulci\vacationsforyou-ai-support-assistant> cd backend
PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\dulci\vacationsforyou-ai-support-assistant\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> python -m venv venv
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> .\venv\Scripts\activate
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend>

(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> pip install flask flask-cors openai python-dotenv requests langchain langchain-openai langchain-community langchain-chroma chromadb pypdf

Downloading xhash-3.7.0-cp314-cp314-win-amd64.whl (32 kB)
Using cached zipp-3.23.1-py3-none-any.whl (10 kB)
Downloading zstandard-0.25.0-cp314-cp314-win-amd64.whl (516 kB)
Using cached flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Using cached importlib_resources-7.1.0-py3-none-any.whl (37 kB)
Using cached pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Using cached requests_oauthlib-2.0.0-py2.py3-none-any.whl (24 kB)
Using cached oauthlib-3.3.1-py3-none-any.whl (160 kB)
Installing collected packages: pypika, flatbuffers, durationpy, zstandard, zipp, xhash, websockets, websocket-client, uuid-utils, tenacity, six, shellin-gham, regex, pyyaml, pyproject hooks, pypdf, pygments, pybase64, protobuf, propcache, packaging, overrides, ormsgpack, orjson, oauthlib, numpy, mpy-extensions, multidict, mmh3, mdurl, langchain-protocol, jsonpointer, importlib-resources, httpx-sse, httpxtools, hf-xet, grpcio, greenlet, fsspec, frozenlist, filelock, bcrypt, annotated-doc, aiohappyeyeballs, yarl, watchfiles, uvicorn, typing-inspect, tiktoken, SQLAlchemy, requests-toolbelt, requests-oauthlib, python-dateutil, opentelemetry-proto, onnxruntime, marshmallow, markdown-it-py, jsonpatch, importlib-metadata, googleapis-common-protos, build, aiosignal, al, rich, pydantic-settings, opentelemetry-exporter-otlp-proto-common, opentelemetry-api, langsmith, langgraph-sdk, kubernetes, dataclasses-json, aiohttp, typer, opentelemetry-semantic-conventions, langchain-core, opentelemetry-sdk, langgraph-checkpoint, langchain-text-splitters, langchain-openai, huggingface-hub, tokenizers, opentelemetry-exporter-otlp-proto-grpc, langgraph-prebuilt, langchain-classic, langgraph, langchain-community, chromadb, langchain-chroma, langchain
Successfully installed SQLAlchemy-2.0.49 aiohappyeyeballs-2.6.1 aiohttp-3.13.5 aiosignal-1.4.0 annotated-doc-0.0.4 bcrypt-5.0.0 build-1.5.0 chromadb-1.5.9 dataclasses-json-0.6.7 durationpy-0.10 filelock-3.29.0 flatbuffers-25.12.19 frozenlist-1.8.0 fsspec-2026.4.0 googleapis-common-protos-1.74.0 greenlet-3.5.0 grpcio-1.80.0 hf-xet-1.5.0 httpxtools-0.7.1 httpx-sse-0.4.3 huggingface-hub-1.14.0 importlib-metadata-8.7.1 importlib-resources-7.1.0 jsonpatch-1.33 jsonpointer-3.1.1 kubernetes-35.0.0 langchain-1.2.17 langchain-chroma-1.1.0 langchain-classic-1.0.6 langchain-community-0.4.1 langchain-core-1.3.3 langchain-openai-1.2.1 langchain-protocol-0.0.15 langchain-text-splitters-1.1.2 langgraph-1.1.10 langgraph-checkpoint-4.0.3 langgraph-prebuilt-1.0.13 langgraph-sdk-0.3.14 langsmith-0.8.2 markdown-it-py-4.1.0 marshmallow-3.26.2 mdurl-0.1.2 mmh3-5.2.1 multidict-6.7.1 mpy-extensions-1.1.0 numpy-2.4.4 oauthlib-3.3.1 onnxruntime-1.25.1 opentelemetry-api-1.41.1 opentelemetry-exporter-otlp-proto-common-1.41.1 opentelemetry-exporter-otlp-proto-grpc-1.41.1 opentelemetry-proto-1.41.1 opentelemetry-sdk-1.41.1 opentelemetry-semantic-conventions-0.62b1 orjson-3.11.9 ormsgpack-1.12.2 overrides-7.7.0 packaging-26.2 propcache-0.4.1 protobuf-6.33.6 pybase64-1.4.3 pydantic-settings-2.14.0 pygments-2.20.0 pypdf-6.10.2 pypika-0.51.1 pyproject hooks-1.2.0 python-dateutil-2.9.0.post0 pyyaml-6.0.3 regex-2026.4.4 requests-oauthlib-2.0.0 requests-toolbelt-1.0.0 rich-15.0.0 shellingham-1.5.4 six-1.17.0 tenacity-9.1.4 tiktoken-0.12.0 toml-0.10.2 tokenizers-0.23.1 typer-0.25.1 typing-inspect-0.9.0 uuid-utils-0.14.1 uvicorn-0.46.0 watchfiles-1.1.1 websocket-client-1.9.0 websockets-16.0 xhash-3.7.0 yarl-1.23.0 zipp-3.23.1 zstandard-0.25.0

[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend>
```

3.3 Configure Environment Variables

8. In the backend folder, create a new file called .env
9. Copy the contents of .env.example into .env
10. Fill in your real values:

```
OPENAI_API_KEY=sk-your-openai-key-here
STREAMLINE_TOKEN_KEY= your_token_key_here
STREAMLINE_TOKEN_SECRET= your_token_secret_here
STREAMLINE_BASE_URL=
```

.env must never be committed to GitHub. Confirm it appears in .gitignore before pushing.

3.4 Load the Call Center Manual (RAG Setup)

11. Obtain VFYmanual.pdf from the previous team lead.
12. Place it in the backend/ folder.
13. Run the ingest script:

```
py ingest.py
```

You should see: Loading manual... Loaded X pages. Created X chunks. Building Vector Store... Vector store built and saved. Manual successfully loaded into vector stores!

You only need to run ingest.py once per machine, or whenever VFYmanual.pdf is updated.

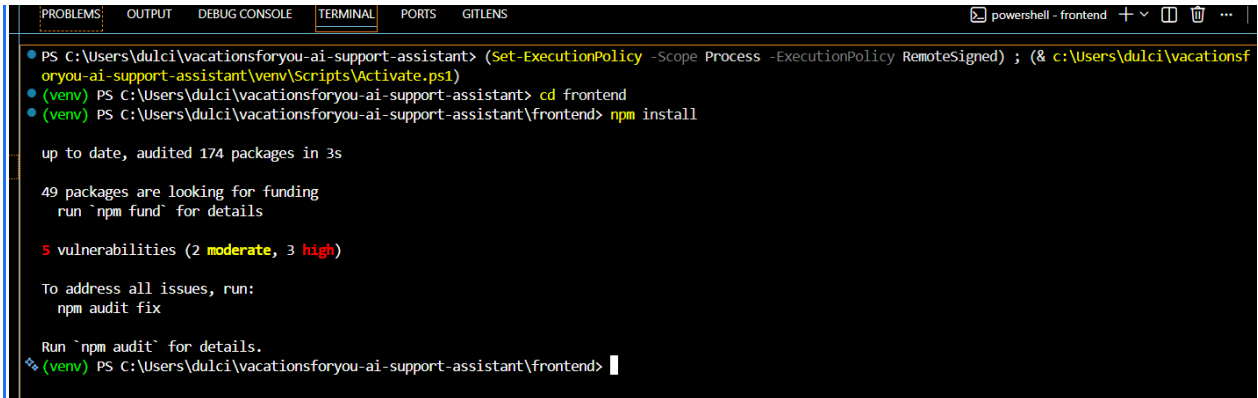
3.5 Set Up the Frontend

14. Open the second terminal window.

15. Navigate to the frontend folder:

```
cd frontend
```

16. Install Node dependencies:

A screenshot of a Visual Studio Code terminal window. The terminal title bar shows 'powershell - frontend'. The terminal content shows a series of commands and their outputs. The first command is a PowerShell command to set the execution policy. The second command is 'cd frontend'. The third command is 'npm install', which outputs 'up to date, audited 174 packages in 3s'. It then shows '49 packages are looking for funding' and 'run `npm fund` for details'. It also reports '5 vulnerabilities (2 moderate, 3 high)' and suggests running 'npm audit fix'. Finally, it prompts to run 'npm audit' for details. The prompt '(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\frontend>' is visible at the bottom of the terminal window.

```
PS C:\Users\dulci\vacationsforyou-ai-support-assistant> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\dulci\vacationsforyou-ai-support-assistant\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant> cd frontend
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\frontend> npm install

up to date, audited 174 packages in 3s

49 packages are looking for funding
  run `npm fund` for details

5 vulnerabilities (2 moderate, 3 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\frontend>
```

```
npm install
```


4. Running the Application

You need two terminals running simultaneously — one for the backend and one for the frontend.

4.1 Start the Backend

In your backend terminal with (venv) active:

```
py chatbot.py
```

You should see:

```
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> py chatbot.py
C:\Users\dulci\vacationsforyou-ai-support-assistant\backend\venv\Lib\site-packages\langchain_core\api\deprecation.py:25: UserWarning: Core Pydantic V1 f
unctionality isn't compatible with Python 3.14 or greater.
  from pydantic.v1.fields import FieldInfo as FieldInfoV1
Vector store loaded successfully!
* Serving Flask app 'chatbot'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
C:\Users\dulci\vacationsforyou-ai-support-assistant\backend\venv\Lib\site-packages\langchain_core\api\deprecation.py:25: UserWarning: Core Pydantic V1 f
unctionality isn't compatible with Python 3.14 or greater.
  from pydantic.v1.fields import FieldInfo as FieldInfoV1
Vector store loaded successfully!
* Debugger is active!
* Debugger PIN: 931-081-769
Vector store loaded successfully! * Serving Flask app 'chatbot' * Running on
http://127.0.0.1:5000
```

Keep this terminal open. If you close it the chatbot will stop working.

4.2 Start the Frontend

In your frontend terminal:

```
npm run dev
```

You should see:

```
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\frontend> npm install
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\frontend> npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.0.0 ready in 732 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

VITE v8.0.0 ready in 288 ms   Local:   http://localhost:5173/
```

4.3 Open the Application

17. Open your browser and go to <http://localhost:5173>
18. You should see a blank page with a  button in the corner.
19. Click the  button to open the chat widget.
20. You should see the welcome message: "Welcome to Vacations For You!"



Vacations For YOU

AI Support Agent – here with all your travel needs

Welcome to Vacations For You! Ask about bookings, properties, check-in times, or cancellations. I'm here to help with all your travel needs.



04:33 PM

Write a message...

Send



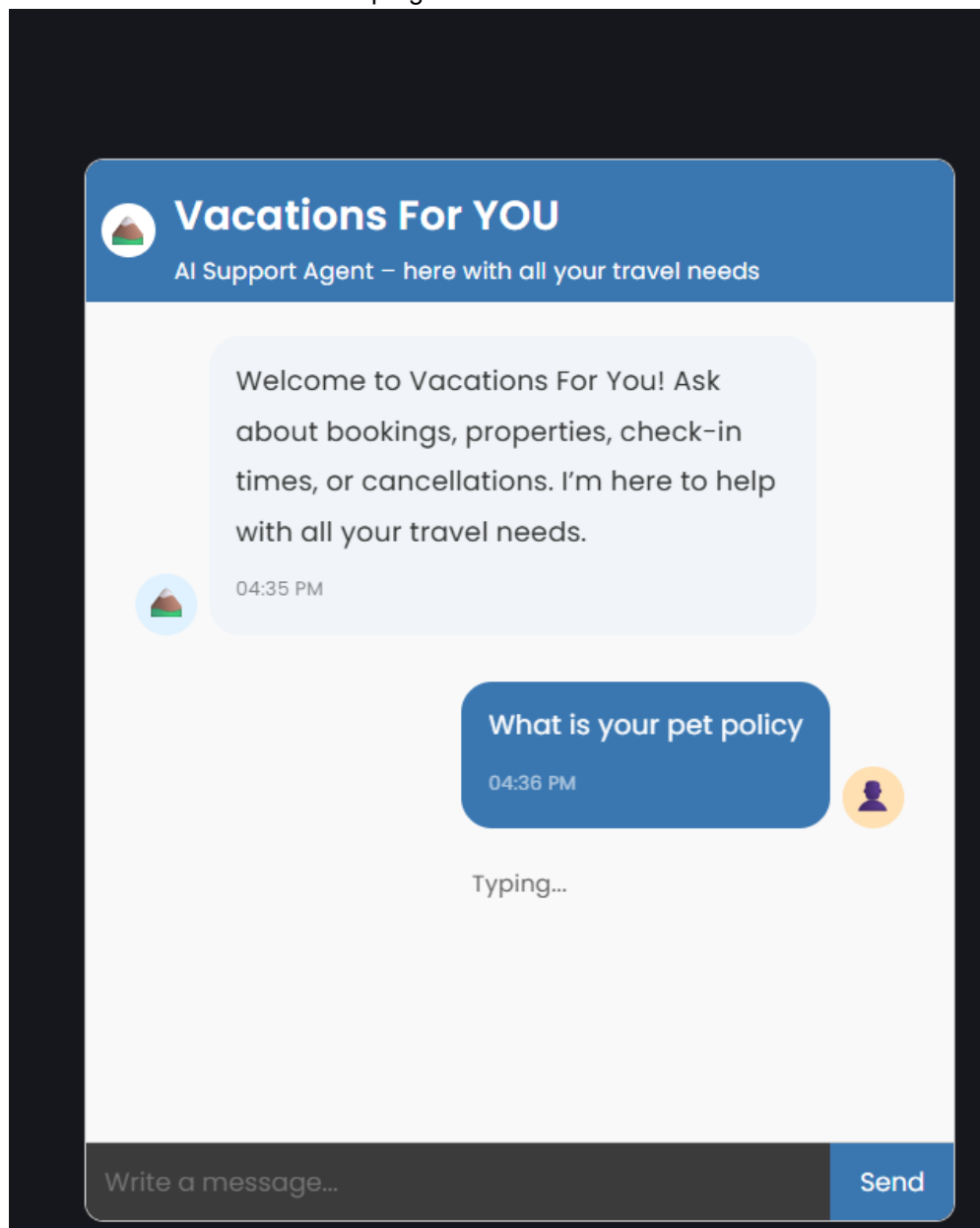
5. Using the Chatbot

5.1 Asking a General Question

21. Type a question in the input bar at the bottom of the chat.
22. Press Enter or click Send.
23. The bot will respond within a few seconds. A 'Typing...' indicator appears while it processes.

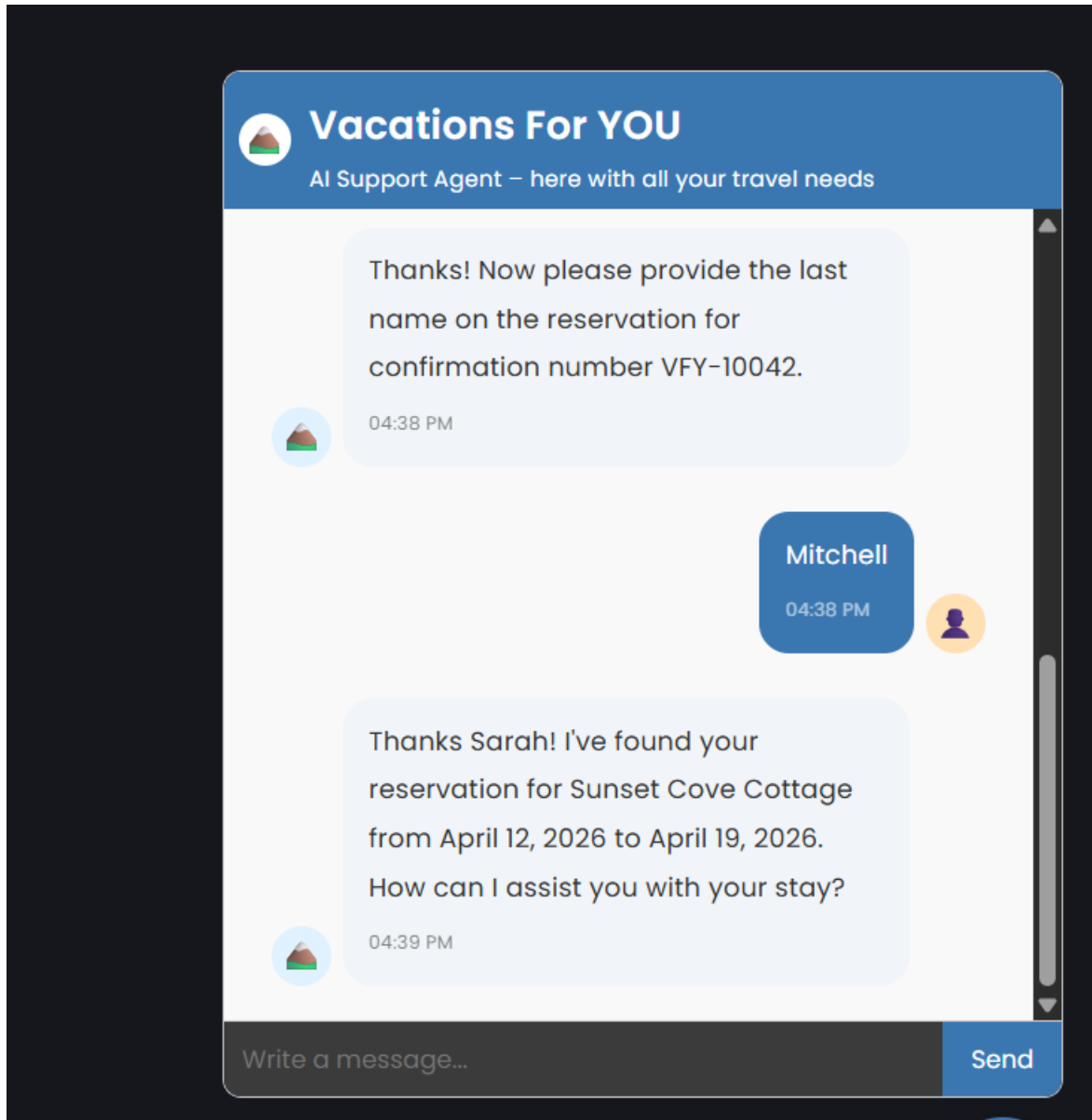
Example questions to try:

- Do you have cabins in Tennessee?
- What is your cancellation policy?
- Are pets allowed?
- What is there to do near your Georgia properties?
- How does the rewards program work?



5.2 Looking Up a Reservation

24. Type: I have a reservation
25. The bot will ask for your confirmation number.
26. Enter one of the test confirmation numbers (see Section 7).
27. The bot will ask for the last name on the reservation.
28. Enter the matching last name.
29. The bot will confirm the reservation and personalize all further responses.



Confirmation numbers are case insensitive. VFY-10042 and vfy-10042 both work. Last names are also case insensitive.

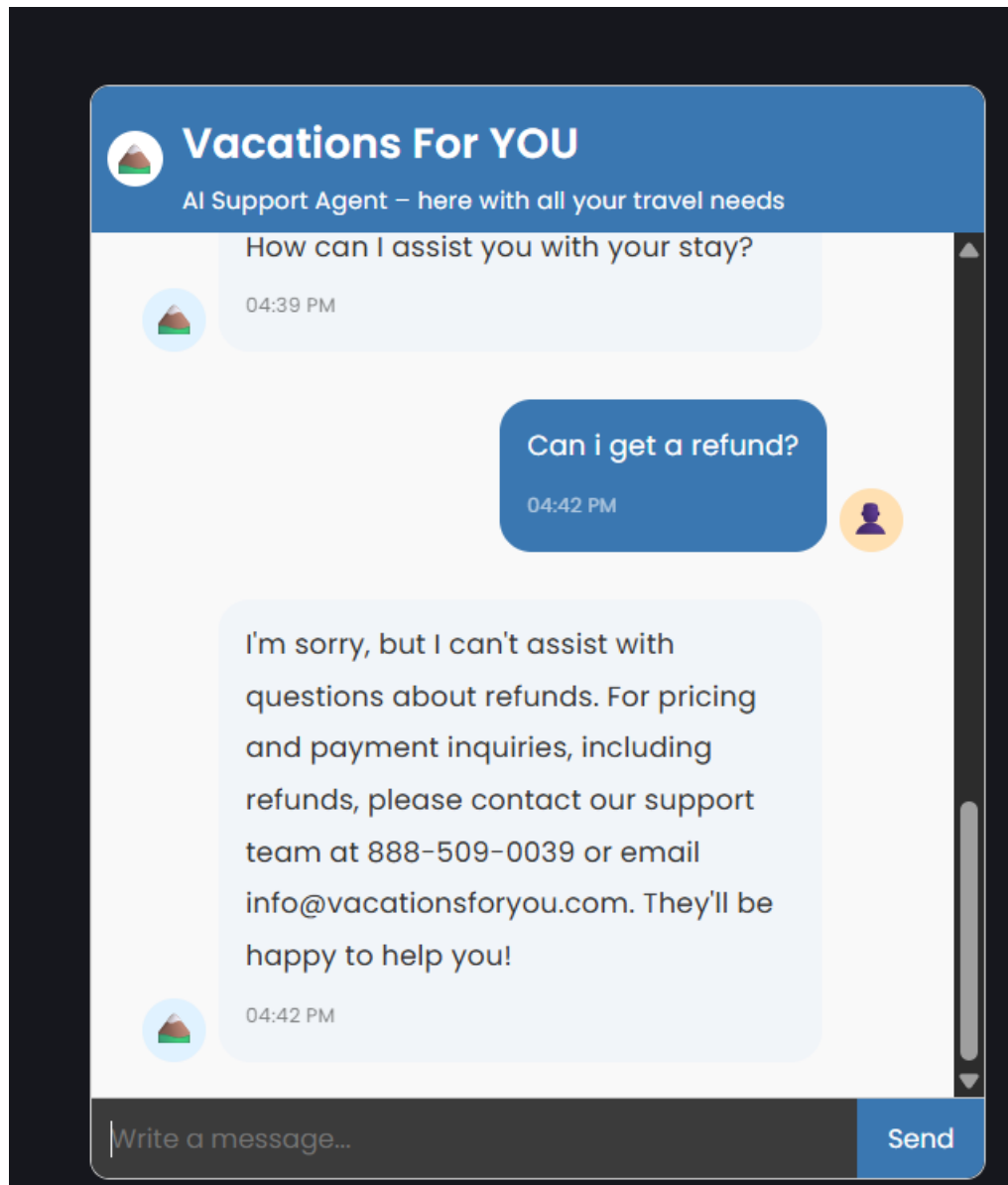
5.3 Out-of-Scope Questions

Some questions are outside the chatbot's scope. Try these to see escalation in action:

- Can I get a refund?

- Can you cancel my reservation?
- How much does it cost to book?

The bot will politely decline and provide the support team with contact information.



5.4 Session Timeout

Sessions automatically expire after 30 minutes of inactivity. If a guest sends a message after the timeout, they will receive a session expired and need to start a new conversation.

6. Viewing Logs and Sessions

6.1 Terminal Logs

Every interaction is printed to the backend terminal as a JSON log entry in real time. Each entry contains:

- timestamp — when the message was sent (UTC)
- session_id — unique ID for this conversation
- user_message — what the guest typed
- bot_reply — what the bot responded
- confidence — HIGH_CONFIDENCE, PARTIAL_CONFIDENCE, or OUT_OF_SCOPE

```
(venv) PS C:\Users\dulci\vacationsforyou-ai-support-assistant\backend> py chatbot.py
e-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
if (datetime.utcnow() - last_active).total_seconds() > time_out minutes * 60:
C:\Users\dulci\vacationsforyou-ai-support-assistant\backend\chatbot.py:125: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  session["last_active"] = datetime.utcnow()
C:\Users\dulci\vacationsforyou-ai-support-assistant\backend\chatbot.py:84: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  timestamp = datetime.utcnow().isoformat()
LOG: {"timestamp": "2026-05-06T20:35:03.761348", "session_id": "dee3953f-a1a2-4b7a-8d3c-11f3c56087f3", "user_message": "Are pets allowed?", "bot_reply": "Pet policies vary by property and destination. In the Alabama Gulf Coast, for example, there are pet-friendly options available. For specific information about a particular property, I recommend checking the listing on our website or contacting our support team at 888-509-0039 or info@vacationsforyou.com for assistance. We're here to help!", "confidence": "HIGH_CONFIDENCE"}
CONTEXT USED: ...
127.0.0.1 - - [06/May/2026 16:35:03] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:35:29] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:35:32.753495", "session_id": "dee3953f-a1a2-4b7a-8d3c-11f3c56087f3", "user_message": "What is your cancellation policy?", "bot_reply": "I'm sorry, but I don't have information about our cancellation policy. For detailed assistance regarding cancellations, please contact our support team at 888-509-0039 or email us at info@vacationsforyou.com. They'll be happy to help you!", "confidence": "OUT_OF_SCOPE"}
CONTEXT USED: ...
127.0.0.1 - - [06/May/2026 16:35:32] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:36:23] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:36:27.622965", "session_id": "05c71475-6261-4b72-b1d4-4a93c924e8c6", "user_message": "What is your pet policy", "bot_reply": "Our pet policy varies by property. Many of our vacation rentals, especially along the Alabama Gulf Coast, offer pet-friendly options. However, specific pet policies, including any associated fees, can differ from one property to another. For detailed information about the pet policy for a specific property, I recommend checking the property listing on our website or contacting our support team at 888-509-0039 or info@vacationsforyou.com. They'll be happy to assist you!", "confidence": "HIGH_CONFIDENCE"}
CONTEXT USED: ...
127.0.0.1 - - [06/May/2026 16:36:27] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:38:38] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:38:38.954533", "session_id": "f4878618-d9a8-420a-98bf-d6f3a139ff53", "user_message": "I have a reservation", "bot_reply": "Sure! Please share your confirmation number and I'll pull up your reservation.", "confidence": "HIGH_CONFIDENCE"}
127.0.0.1 - - [06/May/2026 16:38:38] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:38:49] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:38:50.095730", "session_id": "f4878618-d9a8-420a-98bf-d6f3a139ff53", "user_message": "VFY-10042", "bot_reply": "Thanks! Now please provide the last name on the reservation for confirmation number VFY-10042.", "confidence": "HIGH_CONFIDENCE"}
127.0.0.1 - - [06/May/2026 16:38:50] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:38:59] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:39:00.058345", "session_id": "f4878618-d9a8-420a-98bf-d6f3a139ff53", "user_message": "Mitchell", "bot_reply": "Thanks Sarah! I've found your reservation for Sunset Cove Cottage from April 12, 2026 to April 19, 2026. How can I assist you with your stay?", "confidence": "HIGH_CONFIDENCE"}
127.0.0.1 - - [06/May/2026 16:39:00] "POST /chat HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2026 16:42:03] "OPTIONS /chat HTTP/1.1" 200 -
LOG: {"timestamp": "2026-05-06T20:42:05.564947", "session_id": "f4878618-d9a8-420a-98bf-d6f3a139ff53", "user_message": "Can I get a refund?", "bot_reply": "I'm sorry, but I can't assist with questions about refunds. For pricing and payment inquiries, including refunds, please contact our support team at 888-509-0039 or email info@vacationsforyou.com. They'll be happy to help you!", "confidence": "OUT_OF_SCOPE"}
CONTEXT USED: ...
127.0.0.1 - - [06/May/2026 16:42:05] "POST /chat HTTP/1.1" 200 -
```

6.2 Context Used Log

For policy questions, the terminal also prints a CONTEXT USED: line showing the first 200 characters of the manual chunk that was used to generate the response. This confirms that RAG is working correctly.

6.3 Sessions Endpoint

30. Make sure the backend is running.
31. Open a browser and go to: <http://localhost:5000/logs>
32. You will see a JSON response showing all active sessions, their states, reservation data, and conversation histories.

```
localhost:5000/logs
Pretty-print ☐
[
  {
    "history": [],
    "last_active": "Wed, 06 May 2026 20:35:29 GMT",
    "pending_confirmation": null,
    "reservation": null,
    "state": "normal"
  },
  {
    "history": [],
    "last_active": "Wed, 06 May 2026 20:36:23 GMT",
    "pending_confirmation": null,
    "reservation": null,
    "state": "normal"
  },
  {
    "history": [],
    "last_active": "Wed, 06 May 2026 20:42:03 GMT",
    "pending_confirmation": null,
    "reservation": {
      "check_in": "April 12, 2026",
      "check_out": "April 19, 2026",
      "first_name": "Sarah",
      "last_name": "Mitchell",
      "property": "Sunset Cove Cottage"
    },
    "state": "normal"
  }
]
```

7. Test Data

Three mock reservations are built into the system for testing. Use these during development and demos.

Confirmation	Last Name	Property	Check-in	Check-out	Payment Status
VFY-10042	Mitchell	Sunset Cove Cottage	April 12, 2026	April 19, 2026	Paid in Full
VFY-10087	Torres	Blue Ridge Mountain Cabin	May 3, 2026	May 10, 2026	Deposit Paid
VFY-10155	Park	Gulf Shore Villa	June 21, 2026	June 28, 2026	Unpaid

These reservations are hardcoded in chatbot.py inside the waiting_for_last_name state handler. Update them to match real StreamlineVRS data once the live connection is active.

8. StreamlineVRS Connection

8.1 Testing the Connection

33. Make sure your .env has STREAMLINE_TOKEN_KEY, STREAMLINE_TOKEN_SECRET, and STREAMLINE_BASE_URL filled in.

34. Run the connection test:

```
py test_connection.py
```

35. Read the result:

Response	Meaning	Action
Status 200 + data	Connected and working	You are live. No further action needed.
E0010 — Token invalid	Credentials don't match this URL	Contact sponsor for updated token key and secret.
E0012 — IP access denied	Your IP is not whitelisted	Send your IP (from whatismyip.com) to the sponsor to whitelist.
Connection error	Wrong URL or server unreachable	Check STREAMLINE_BASE_URL in .env. URL rotates monthly.

8.2 Monthly Credential Rotation

StreamlineVRS rotates the base URL and credentials monthly. The URL format is https://[month]-web.4vrs.com/api/json (e.g. apr-web, may-web). When you see E0010 or a connection error at the start of a new month:

36. Contact Matthew Shadle at Vacations for YOU for the new token key, token secret, and base URL.

37. Update STREAMLINE_TOKEN_KEY, STREAMLINE_TOKEN_SECRET, and STREAMLINE_BASE_URL in your .env file.

38. Run py test_connection.py again to confirm.

Credentials expire monthly. If the chatbot stops connecting at the start of a new month, this is almost always the cause.

9. Updating the Call Center Manual

When the call center manual is updated, the RAG vector store must be rebuilt. This takes about 2-3 minutes.

39. Obtain the updated manual PDF from the sponsor or team lead.

40. Replace VFYmanual.pdf in the backend/ folder with the new version.

41. Delete the existing chroma_db/ folder:

```
rmdir /s /q chroma_db    (Windows)
rm -rf chroma_db        (Mac/Linux)
```

42. Rebuild the vector store:

```
py ingest.py
```

43. Restart the backend:

```
py chatbot.py
```

44. Run the evaluation to confirm accuracy has not regressed:

```
py evaluate.py
```

The manual PDF and chroma_db/ folder is both gitignored. Never commit them to GitHub.

10. Troubleshooting Common Errors

Error	Cause	Fix
ModuleNotFoundError: No module named 'flask'	venv is not active or packages not installed	Run <code>.\venv\Scripts\activate</code> then <code>pip install flask flask-cors openai python-dotenv requests</code>
Error connecting to server (in browser)	Flask backend is not running	Run <code>py chatbot.py</code> in the backend terminal
Vector store loaded — but RAG answers seem wrong	chroma_db is stale or built from wrong manual	Delete <code>chroma_db/</code> and rerun <code>py ingest.py</code>
E0010 — Token invalid	Credentials don't match the URL	Get updated credentials from sponsor
E0012 — IP access denied	Your IP is not whitelisted	Send IP from whatismyip.com to sponsor
Can't open <code>test_connection.py</code> — file not found	Running from wrong folder	<code>cd backend</code> first then run <code>py .\test_connection.py</code>
<code>npm run dev</code> fails — vite not found	npm install not run	<code>cd frontend</code> then <code>npm install</code>
Chat shows blank — no welcome message	React state not initializing	Hard refresh browser with <code>Ctrl+Shift+R</code>
Session expired immediately	Timeout set too short in <code>chatbot.py</code>	Check <code>timeout_minutes</code> value in <code>/chat</code> route — should be 30

11. Security Checklist

Before committing or pushing to GitHub, verify the following:

Item	Check
.env is in .gitignore	Run git status — .env must NOT appear
.env.example has no real values	Open .env.example — should only contain placeholder text
VFYmanual.pdf is in .gitignore	Run git status — VFYmanual.pdf must NOT appear
chroma_db/ is in .gitignore	Run git status — chroma_db/ must NOT appear
No API keys in any .py or .tsx file	Search the codebase for sk- and token_ to confirm no hardcoded keys
HTTPS used for all external calls	All API calls use https:// URLs

If .env appears in git status, run: `git rm --cached .env` before pushing.

12. Appendix

Project	VacationsForYOU AI Guest Support Assistant
Version	1.0 — Milestone 3
Audience	Next development team
Date	Spring 2026
Repository	github.com/CGrogan4/vacationsforyou-ai-support-assistant
Team	Cassidie Grogan · Kendal Elison · Ezra Begashaw · Benjamin Dulcio · Wilfred Faltz